An introduction to optimal transport

Pierre Geurts

Institut Montefiore, University of Liège, Belgium

INFO8004
Advanced Machine Learning
March 12, 2026

Acknowledgment

These slides are heavily based on the following presentations and reference:

- Rémy Flamary, "Introduction to Computational Optimal Transport", January, 2026.
- Charlotte Bunne and Marco Cuturi, "Optimal transport in learning, control, and dynamical systems", ICML Tutorial, 2023.

Optimal transport

Distributions are everywhere in machine learning. Many objects, datasets can be seen as distributions.

Optimal transport provides tools to compare distributions.

Studied by famous mathematicians over the years:

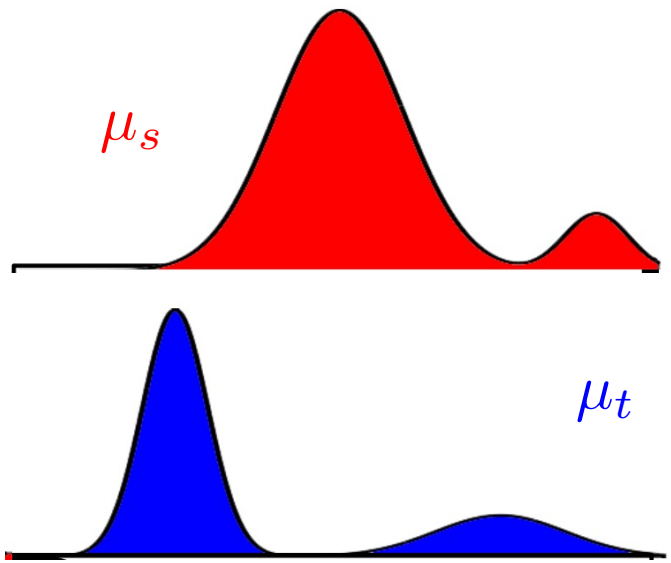
Monge | Kantorovich | Koopmans | Dantzig | Brenier | Otto | McCann | Villani | Figalli

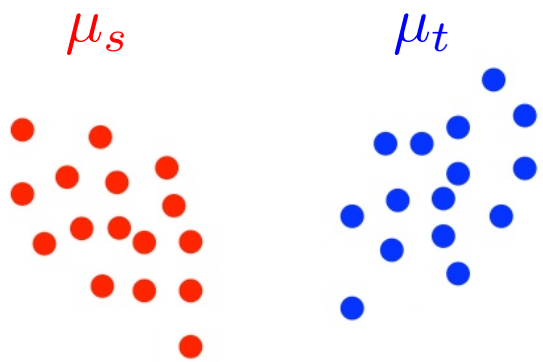
Nobel '75 | Fields '10 | Fields '18

[Credit: Flamary]

Distance between probability measures

How to compute the distance between two distributions μ_s and μ_t?





Standard distances between probability measures

Two standard distance measures between probability measures are L^p norm and Kullback-Leibler divergence:

- L^P norm:

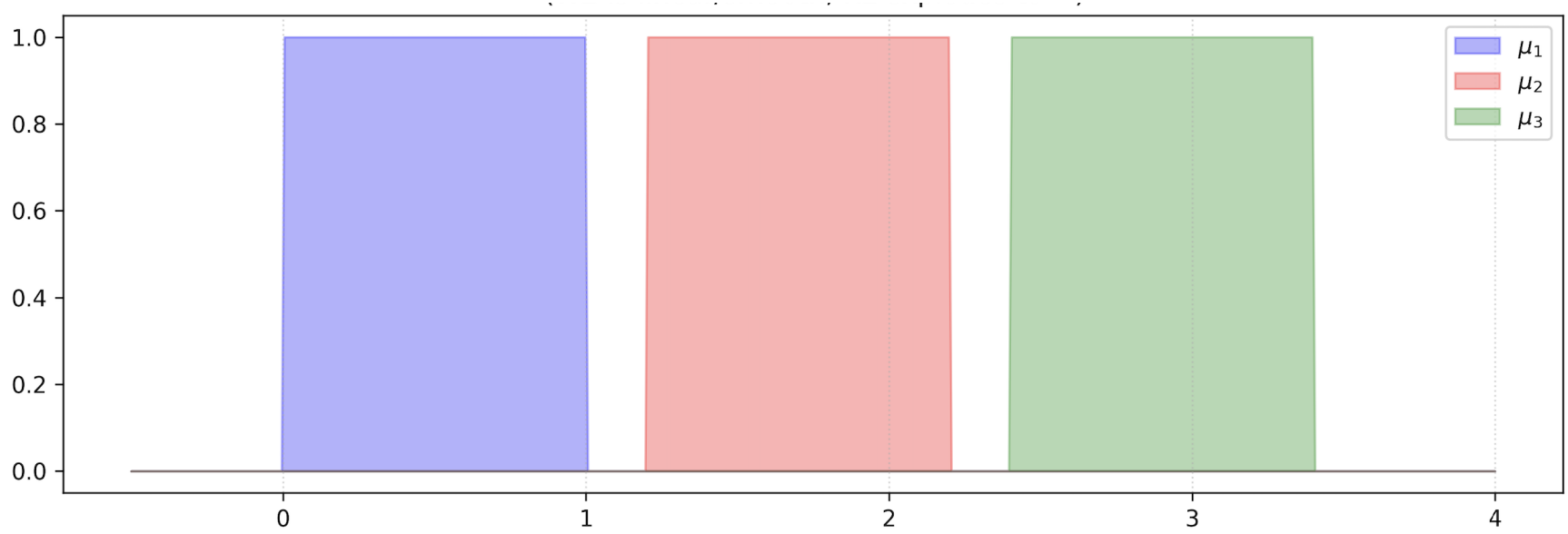
$$||\mu_s - \mu_t||_p = \left(\int |\mu_s(x) - \mu_t(x)|^p dx \right)^{\frac{1}{p}}$$

- KL divergence:

$$KL(\mu_s||\mu_t) = \int \mu_s(x) \log \frac{\mu_s(x)}{\mu_t(x)} dx$$

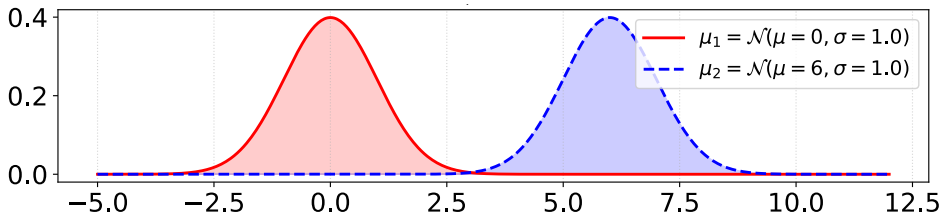
These distances measure overlap, not displacement.

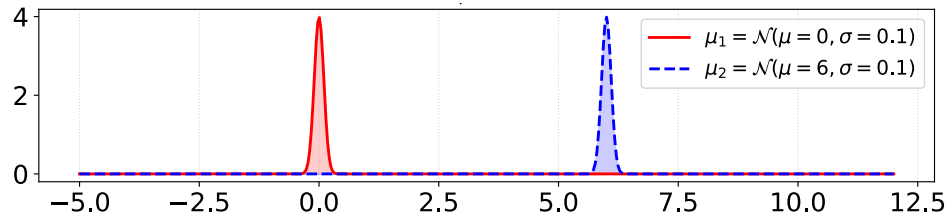
Distance between probability measures

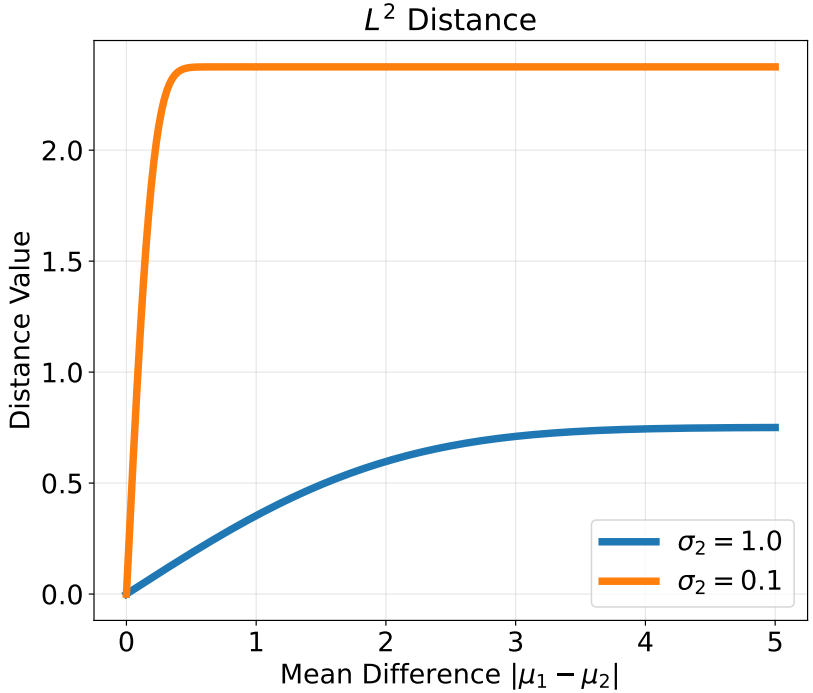


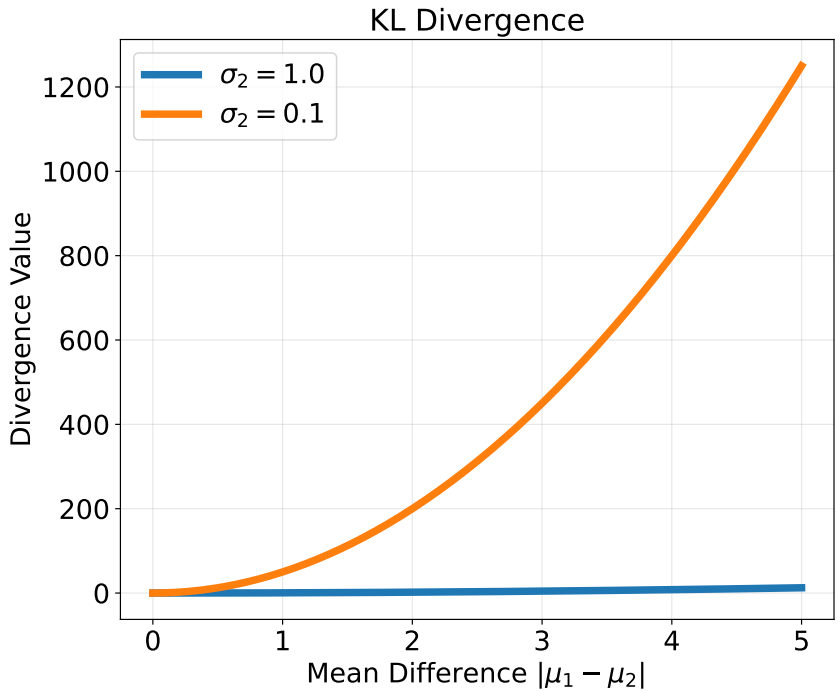
μ_1, μ_2, and μ_3 are all equidistant according to L^p norm and KL divergence.

Standard distances between probability measures





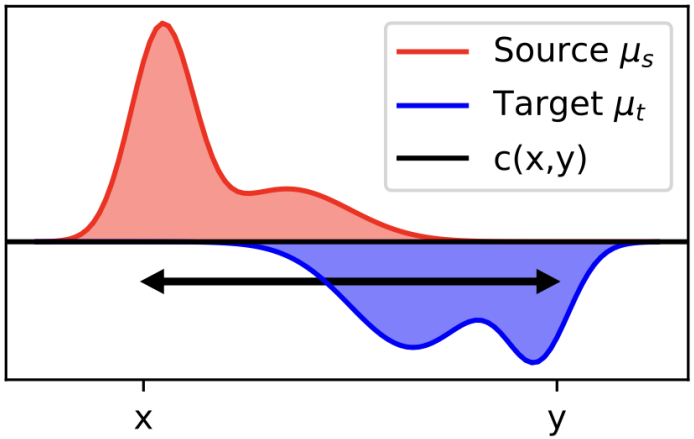


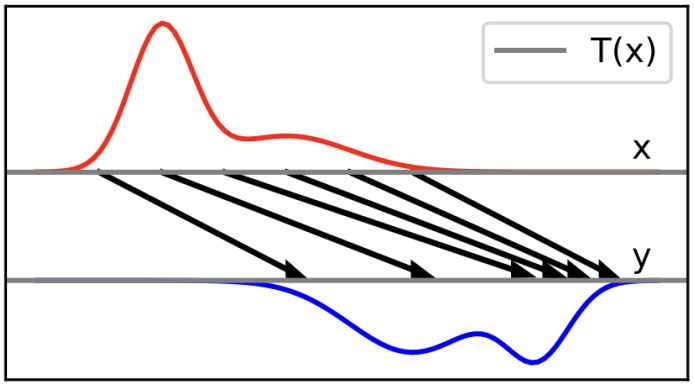


Outline

Outline

OT distance: Monge (1871)'s formulation





- How to move sand from one place (déblais) to another (remblais) while minimizing the effort?
- Find a mapping T between the two distributions of mass (transport)
- Optimize with respect to a displacement cost $c(x, y)$ (optimal)

[Credit: Flamary]

OT distance: Monge (1871)'s formulation

Formally:

- Let μ_s and μ_t the two probability measures to be compared defined on sample spaces Ω_s and Ω_t respectively.
- Let $c : \Omega_s \times \Omega_t \to \boldsymbol{R}^+$ be the displacement cost.
- Monge's OT distance is defined as:

$$OT_{\text{Monge}}(\mu_s, \mu_t) = \inf_{T\#\mu_s = \mu_t} \int_{\Omega_s} c(x, T(x))\mu_s(x)dx,$$

  over all possible mapping $T : \Omega_s \to \Omega_t$ such that $T\#\mu_s = \mu_t$.
- $T\#$ denotes the pushfoward operator. $\nu = T\#\mu$ is the probability measure obtained from μ by the transformation T. For any region $B \subseteq \Omega_\mu$:

$$\nu(B) = \mu(T^{-1}(B))$$

[Credit: Flamary]

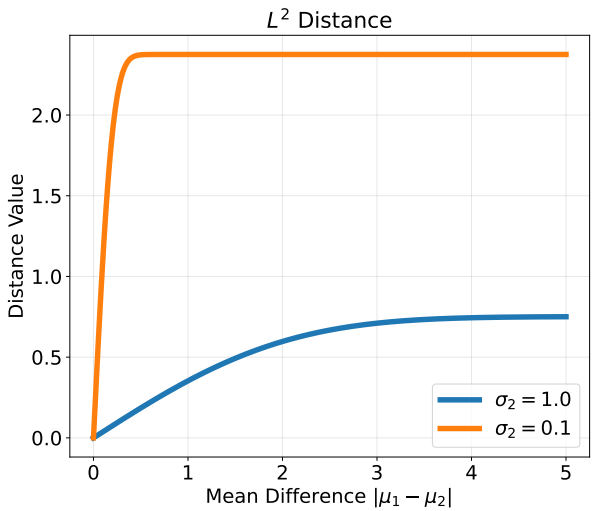
OT distance: Monge (1871)'s formulation

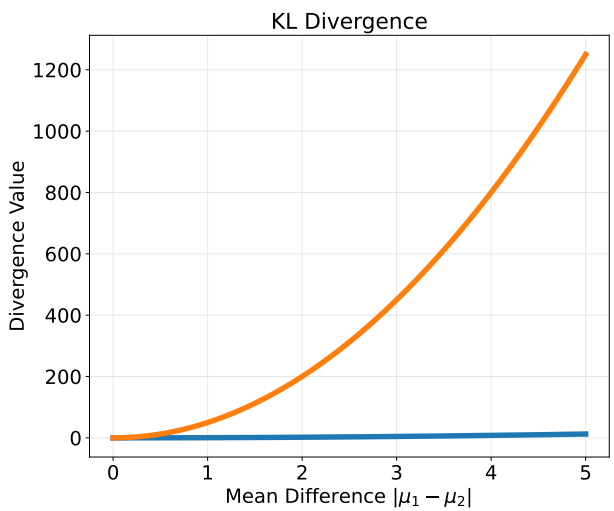
If $\mu_s = \mathcal{N}(m_s, \sigma_s)$ and $\mu_t = \mathcal{N}(m_t, \sigma_t)$ are two univariate Gaussian distributions and using $c(x, y) = (x - y)^2$, one can show that:

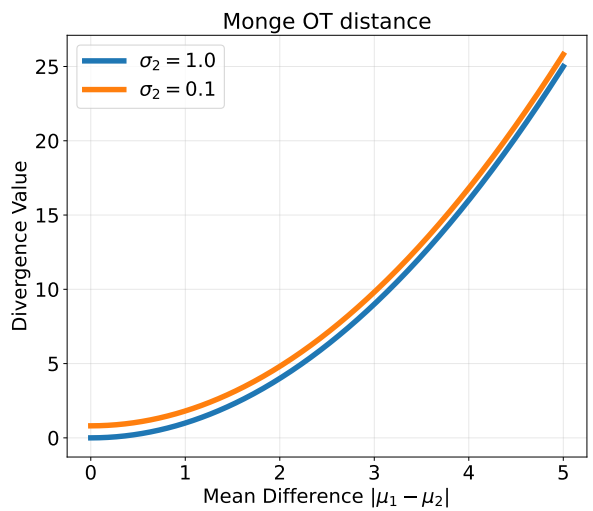
$$OT_{\text{Monge}}(\mu_s, \mu_t) = (m_s - m_t)^2 + (\sigma_s - \sigma_t)^2,$$

and that the transport map $T(x)$ that minimizes the transport cost is:

$$T(x) = m_t + \frac{\sigma_t}{\sigma_s}(x - m_s).$$







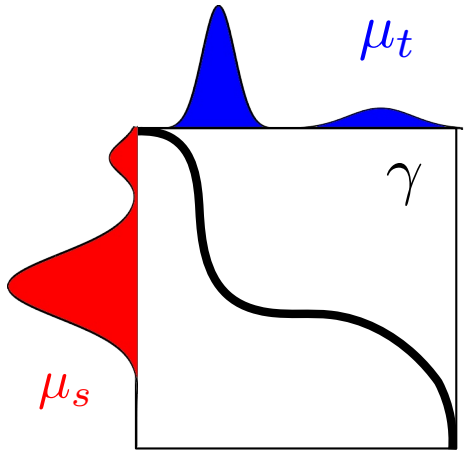
OT distance: Monge (1871)'s formulation

Limitations:

- $OT_{\text{Monge}}(\mu_s, \mu_t)$ is however not always defined. There might not exist any T such that $T\#\mu_s = \mu_t$, in which case OT_{Monge} is undefined.
  - E.g.: $\mu_s = \delta_{y_1}$ and $\mu_t = w_1\delta_{x_1} + w_2\delta_{x_2}$
- Unicity of T is not guaranteed.
- A map T exists and is unique for $c(x, y) = ||x - y||^2$ and continuous distributions (Bernier, 1991).
- $T\#\mu_s = \mu_t$ is a non-convex constraint and thus computing OT_{Monge} is very difficult in general.

$\Rightarrow$ Kantorovich (1942)'s relaxation address these issues.

OT distance: Kantorovich (1942)'s formulation



$$OT_K(\mu_s, \mu_t) = \min_{\gamma \in \mathcal{P}(\mu_s, \mu_t)} \int_{\Omega_s \times \Omega_t} c(x,y)\gamma(x,y)dxdy$$

with

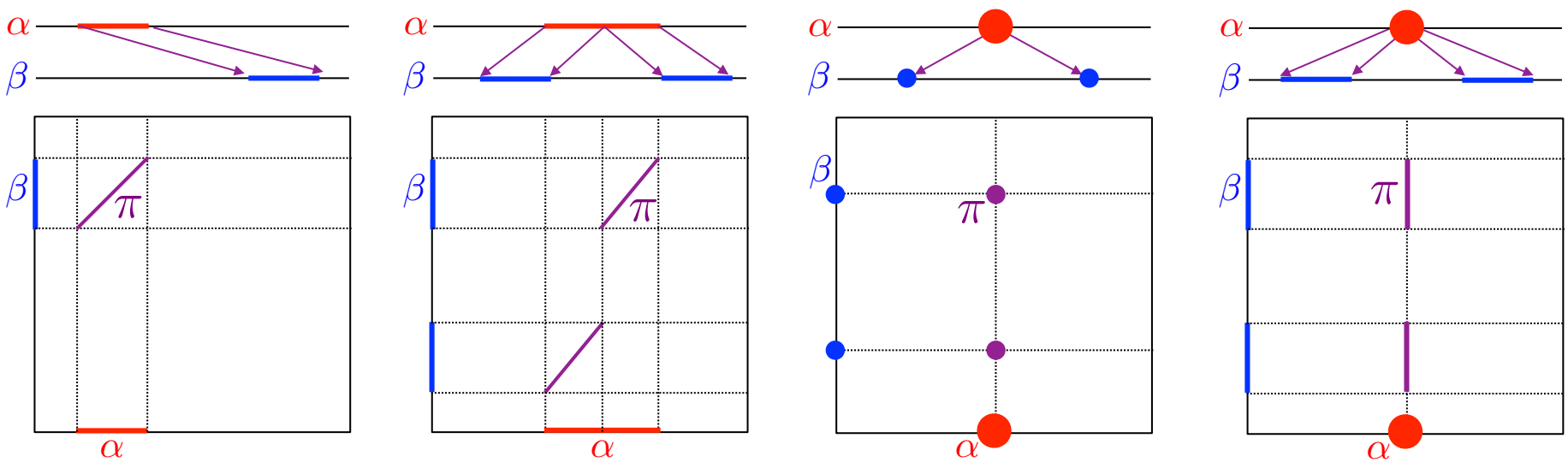
$$\mathcal{P}(\mu_s, \mu_t) = \left\{ \gamma \geq 0, \int_{\Omega_t} \gamma(x,y)dy = \mu_s, \int_{\Omega_s} \gamma(x,y)dx = \mu_t \right\}$$

- The map T is replaced by a **coupling** γ, which is a joint probability measure with marginals μ_s and μ_t.
- A solution always exists since $\gamma(x,y) = \mu_s(x)\mu_t(y) \in \mathcal{P}(\mu_s, \mu_t)$.

OT distance: Kantorovich (1942)'s formulation

$$OT_K(\mu_s, \mu_t) = \min_{\gamma \in \mathcal{P}(\mu_s, \mu_t)} \int_{\Omega_s \times \Omega_t} c(x, y)\gamma(x, y)dxdy$$

Illustration in 1D



[Credit: (Peyré and Cuturi, 2018)]

p-Wasserstein distance

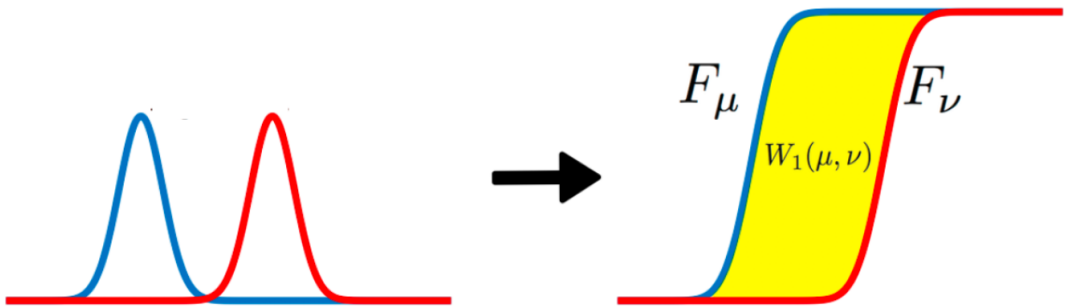
Special case:

$$\begin{aligned} \mathcal{W}_p(\mu_s, \mu_t) &= \min_{\gamma \in \mathcal{P}(\mu_s, \mu_t)} \left(\int_{\Omega_s \times \Omega_t} d(x,y)^p \gamma(x,y) dx dy \right)^{1/p} \\ &= \min_{\gamma \in \mathcal{P}(\mu_s, \mu_t)} \left(E_{(x,y)\sim\gamma}\{d(x,y)^p\} \right)^{1/p}, \end{aligned}$$

where $d(x,y)$ is some distance (e.g. euclidean or geodesic distance)

- $\mathcal{W}_p$ satisfies all axioms of a distance (symetry, separation, and triangular inequality).
- Reduces to $d(x,y)$ when μ_s and μ_t are two diracs at x and y respectively.
- $\mathcal{W}_1$ is called Earth Mover's Distance (Rubner et al., 2000).

Special case: OT in one dimension



- F_μ the cumulative distribution of μ: $F_\mu(t) = \mu(]-\infty, t])$.
- $F_\mu^{-1}(q), q \in [0,1]$ the quantile function:

$$F_\mu^{-1}(q) = \inf\{x \in \boldsymbol{R} : F_\mu(x) \geq q\}.$$

- We have:

$$W_1(\mu_s, \mu_t) = \int_0^1 |F_{\mu_s}^{-1}(q) - F_{\mu_t}^{-1}(q)| dq.$$

Special case: OT between Gaussians

Wasserstein between Gaussian distributions (Bures-Wasserstein):

- $\mu_s \sim \mathcal{N}(m_s, \Sigma_s)$ and $\mu_t \sim \mathcal{N}(m_t, \Sigma_t)$
- OT distance with $c(x, y) = ||x - y||_2^2$ reduces to:

$$\mathcal{W}_2^2(\mu_s, \mu_t) = ||m_s - m_t||_2^2 + \mathcal{B}(\Sigma_s, \Sigma_t)^2$$

where B is the Bures metric:

$$\mathcal{B}(\Sigma_s, \Sigma_t)^2 = \mathrm{Tr}\left(\Sigma_s + \Sigma_t - 2(\Sigma_s^{1/2}\Sigma_t\Sigma_s^{1/2})^{1/2}\right)$$

- In 1D, it reduces to:

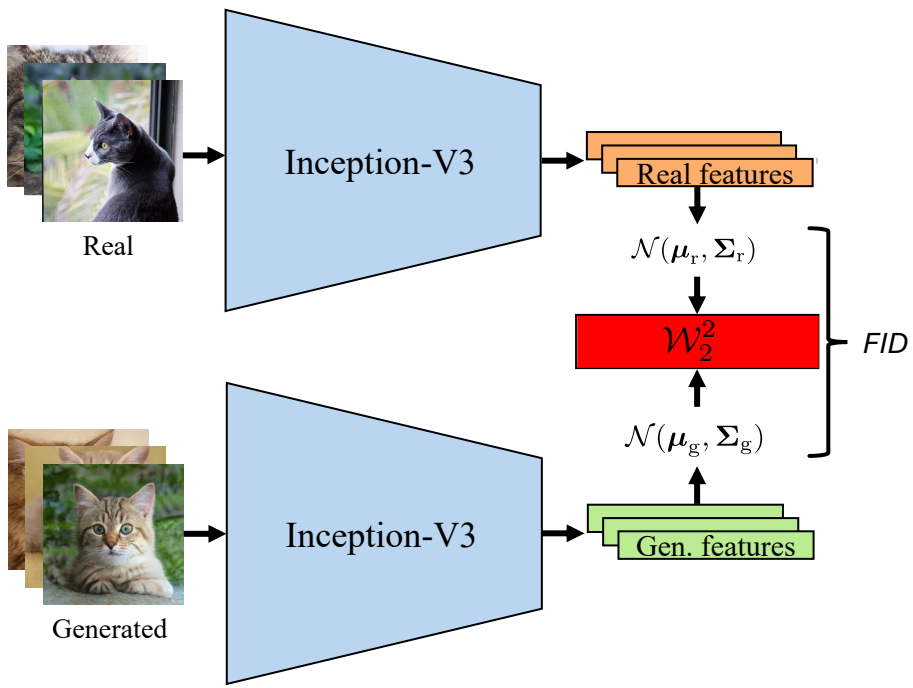
$$\mathcal{W}_2^2(\mu_s, \mu_t) = (m_s - m_t)^2 + (\sigma_s - \sigma_t)^2,$$

which is equal to $OT_{\text{Monge}}(\mu_s, \mu_t)$.

Application: Fréchet inception distance (FID)

Standard evaluation metric to assess image generators:

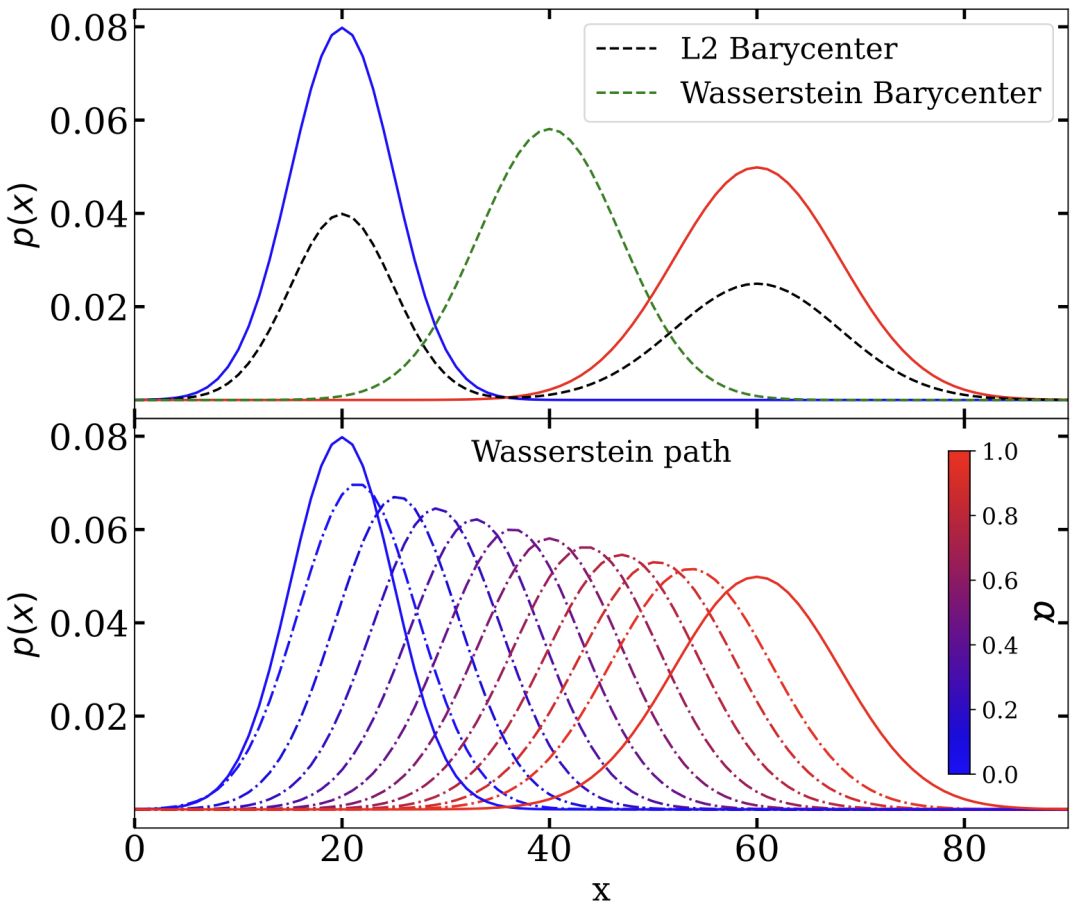
- Pass real and generated images through a pre-trained network
- Fit a multivariate Gaussian distribution on both feature vector sets
- Compute $\mathcal{W}_2^2$ (using Bures-Wasserstein)



[Credit: (Kynkaanniemi et al., 2023)]

Wasserstein barycenters

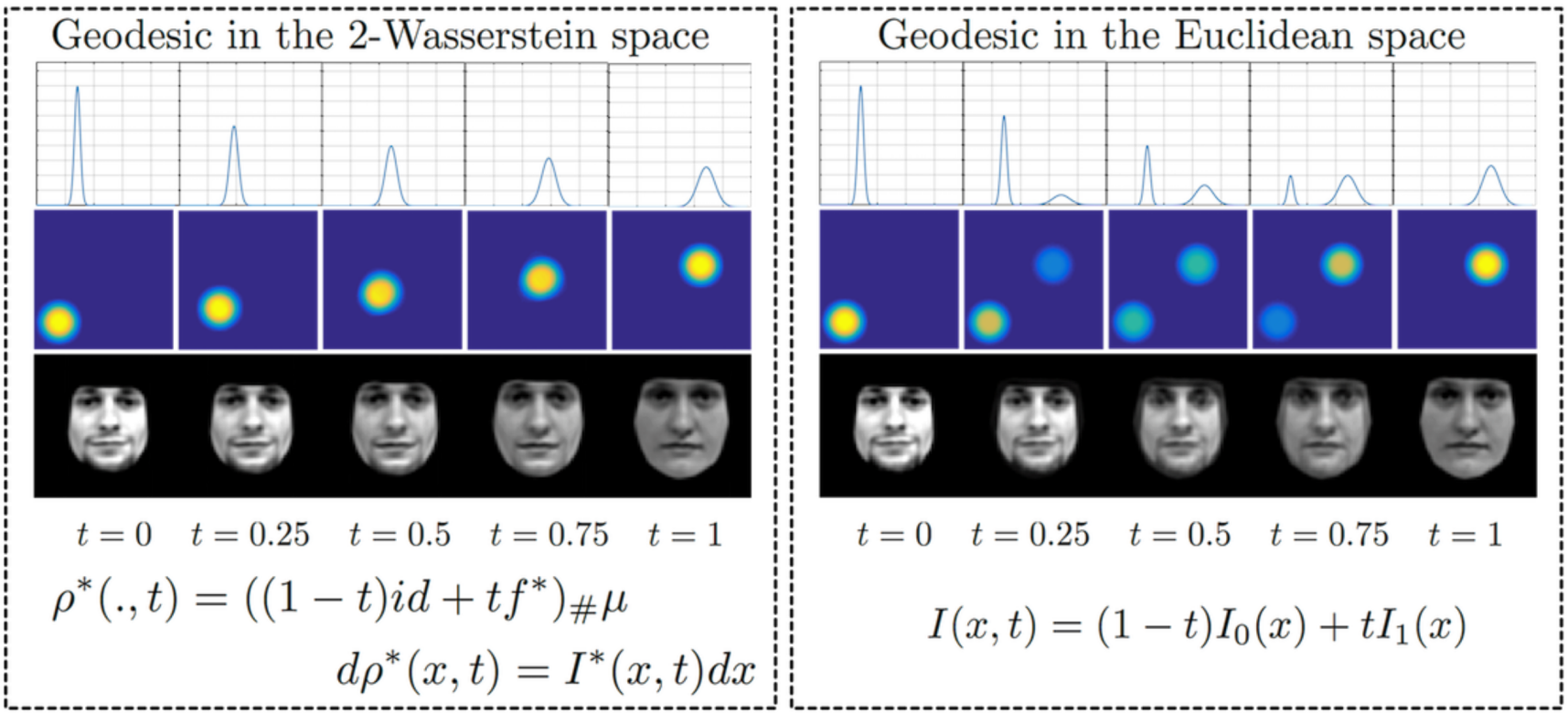
$$\bar{\mu} = \arg\min_{\mu} \sum_{i=1}^{n} \lambda_i \mathcal{W}_p^p(\mu_i, \mu), \text{ with } \lambda_i > 0 \text{ and } \sum_i \lambda_i = 1.$$



[Credit: (Ramirez et al., 2024)]

Wasserstein barycenters

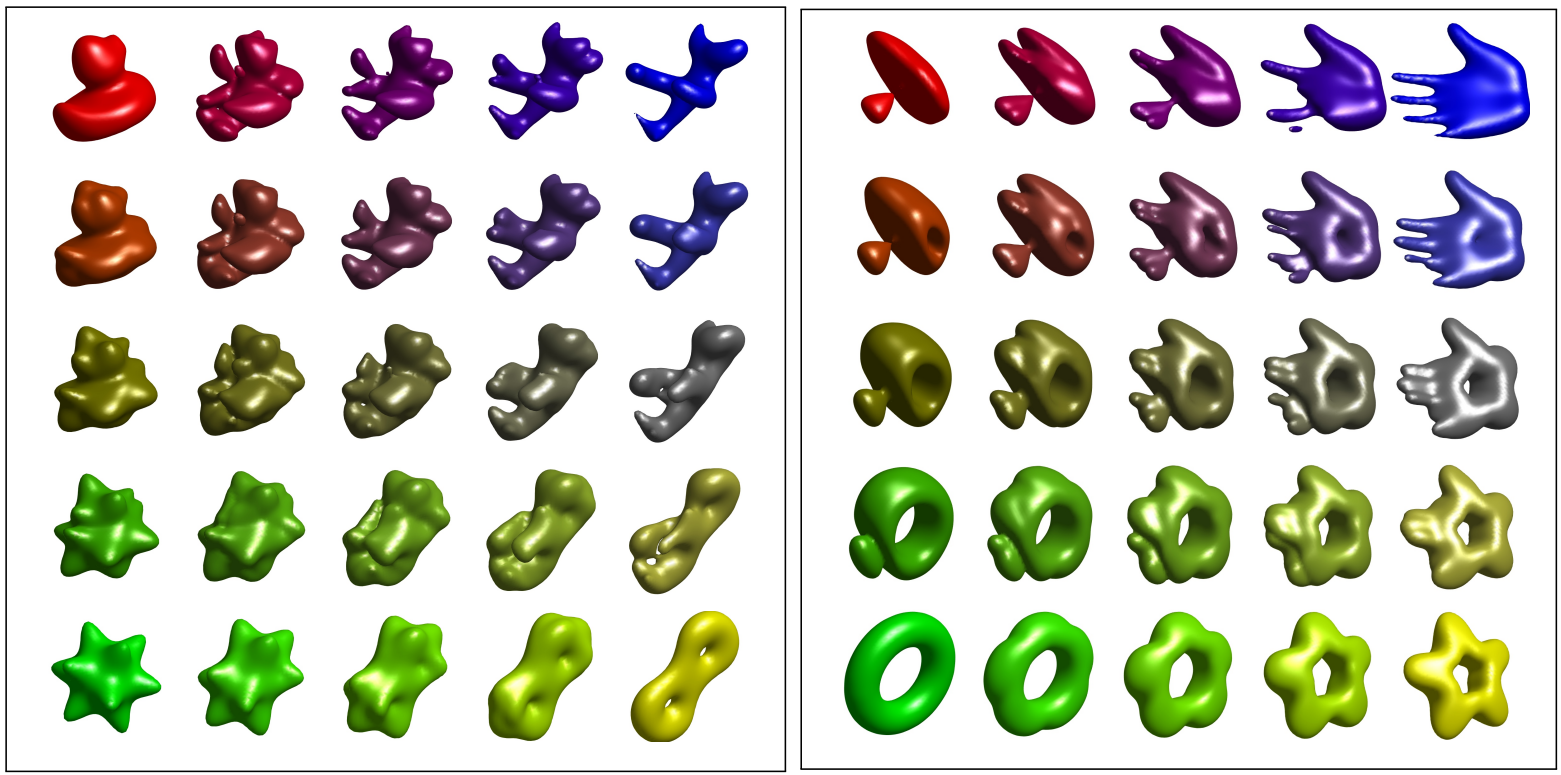
$$\bar{\mu} = \arg\min_{\mu} \sum_{i=1}^{n} \lambda_i \mathcal{W}_p^p(\mu_i, \mu), \text{ with } \lambda_i > 0 \text{ and } \sum_i \lambda_i = 1.$$



[Credit: (Kolouri et al., 2017)]

Wasserstein barycenters

$$\bar{\mu} = \arg\min_{\mu} \sum_{i=1}^{n} \lambda_i \mathcal{W}_p^p(\mu_i, \mu), \text{ with } \lambda_i > 0 \text{ and } \sum_i \lambda_i = 1.$$



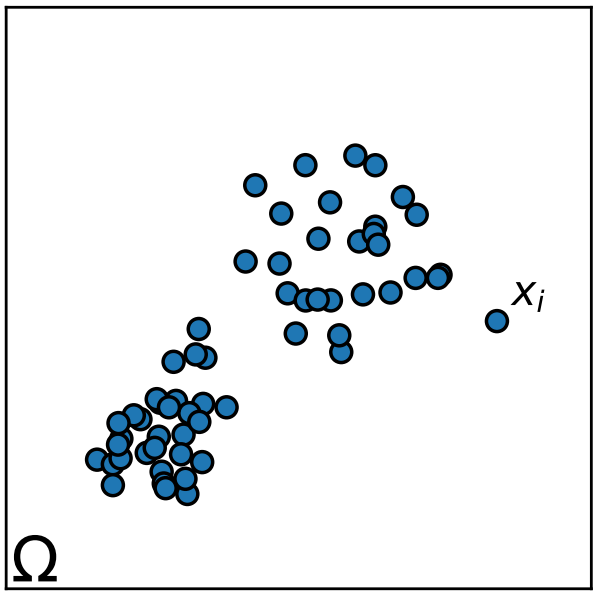
[Credit: (Peyré and Cuturi, 2018)]

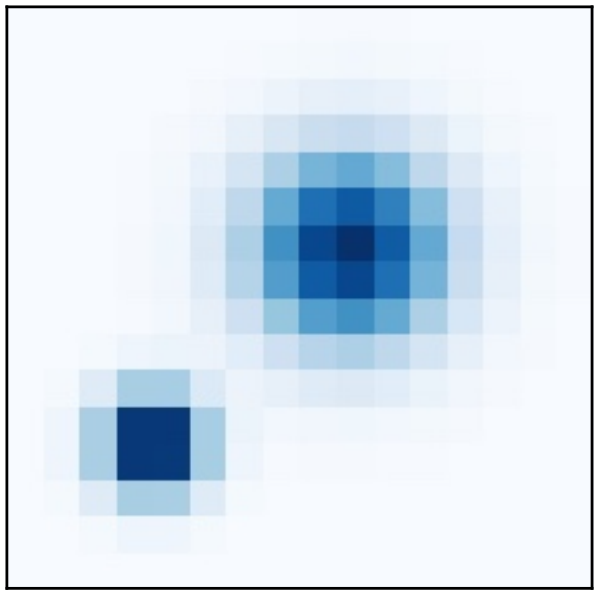
Outline

Discrete distributions: empirical vs histogram

Discrete measure: $\mu = \sum_{i=1}^{n} a_i \delta_{x_i}$, $x_i \in \Omega$, $\sum_{i=1}^{n} a_i = 1$.

Can be used to represent point clouds or histograms





[Credit: Flamary]

Kantorovich's OT with discrete distributions

When $\mu_s = \sum_{i=1}^{n_t} a_i \delta_{x_i^s}$ and $\mu_t = \sum_{i=1}^{n_t} b_i \delta_{x_i^t}$, compute:

$$\mathcal{W}_c(\mu_s, \mu_t) = \min_{\mathrm{T} \in \Pi(\mu_s, \mu_t)} \left\{ \langle \mathrm{T}, \mathrm{C} \rangle_F = \sum_{i,j} T_{ij} C_{ij} \right\},$$

where C is the cost matrix with $C_{ij} = c(x_i^s, x_i^t)$ and the marginal constraints become:

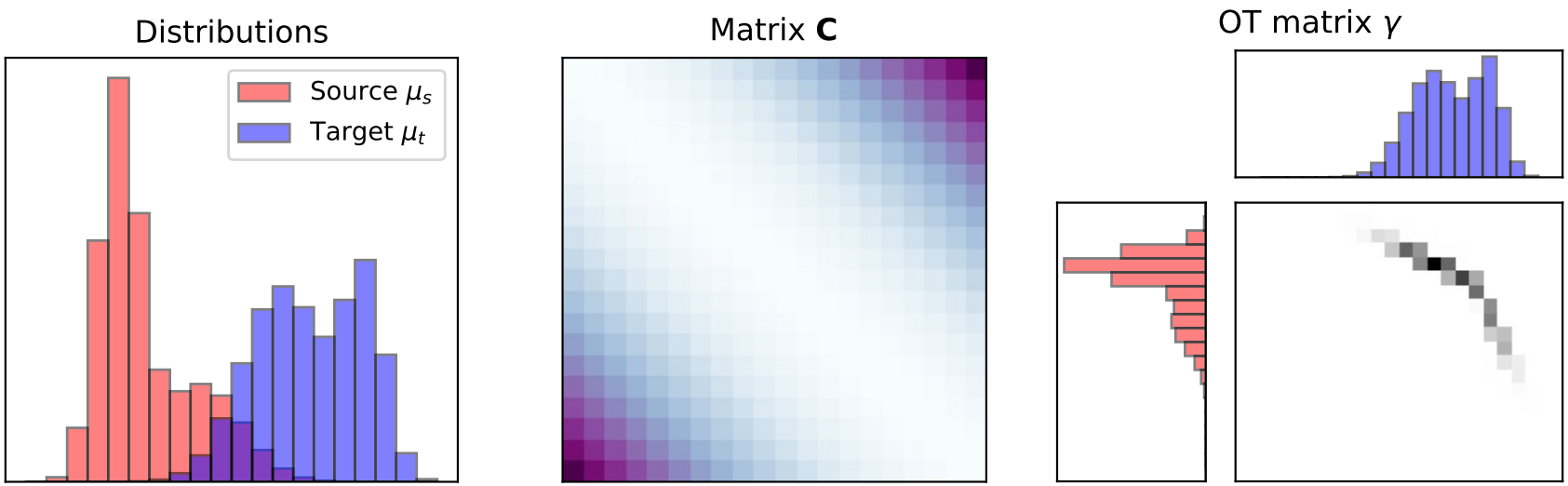
$$\Pi(\mu_s, \mu_t) = \left\{ \mathrm{T} \in (\boldsymbol{R}^+)^{n_s \times n_t} | \mathrm{T} 1_{n_t} = \mathrm{a}, \mathrm{T}^T 1_{n_s} = \mathrm{b} \right\}$$

Two main practical interests:

- Compute the $\mathcal{W}_c$ distance between (empirical) distributions
- Find the optimal coupling (matrix):

$$\mathrm{T}^* = \arg \min_{\mathrm{T} \in \Pi(\mu_s, \mu_t)} \langle \mathrm{T}, \mathrm{C} \rangle .$$

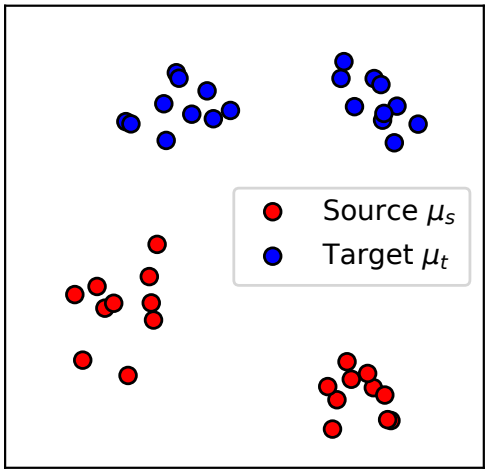
Illustration: histograms



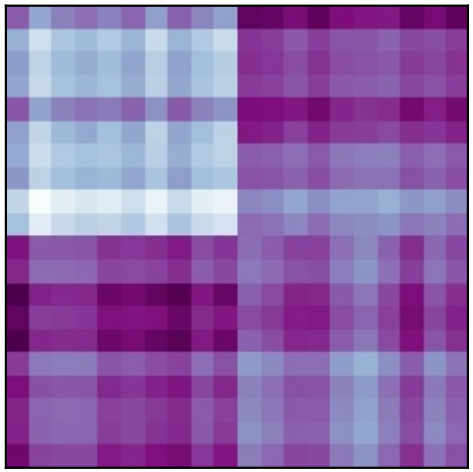
[Credit: Flamary]

Illustration: point clouds

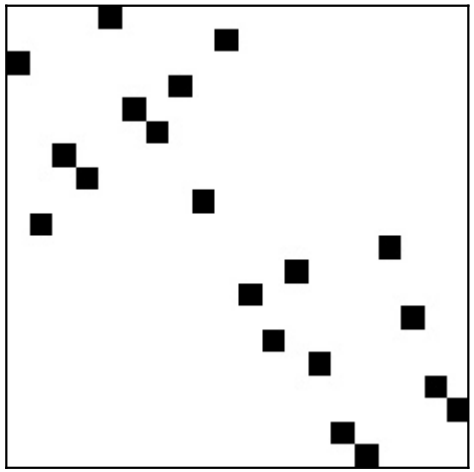
Distributions



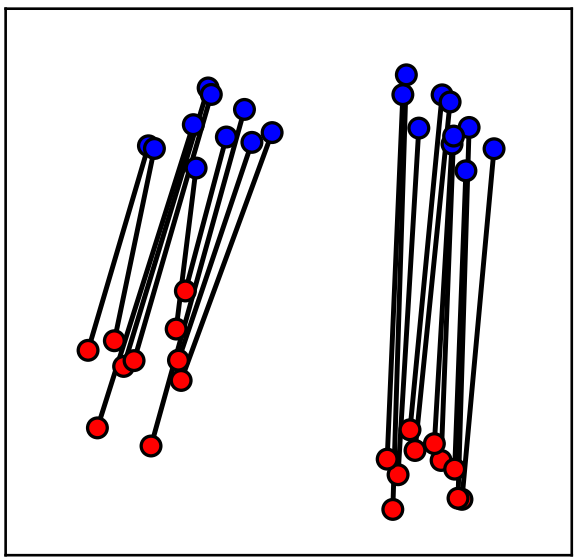
Matrix $\mathbf{C}$



OT matrix γ

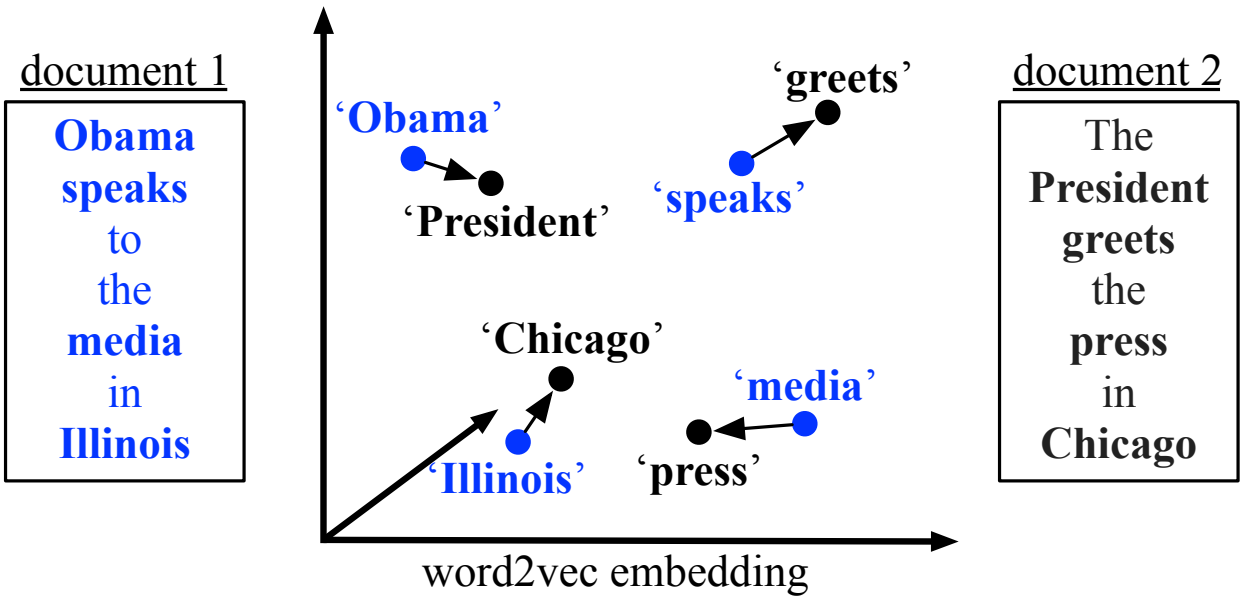


OT matrix with samples



[Credit: Flamary]

Application: word mover's distance



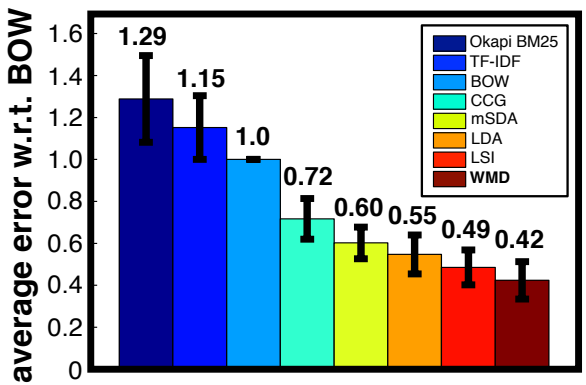
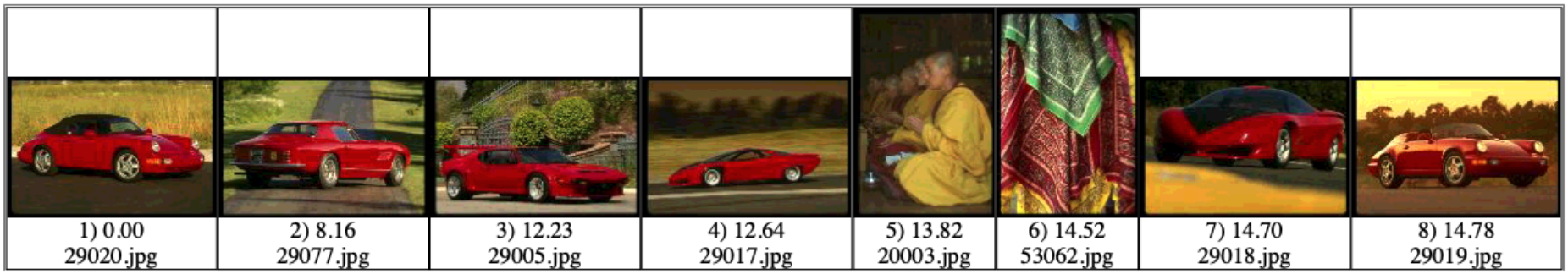


Figure 4. The kNN test errors of various document metrics averaged over all eight datasets, relative to kNN with BOW.

[Credit: (Kusner et al., 2015)]

Application: content-based image retrieval

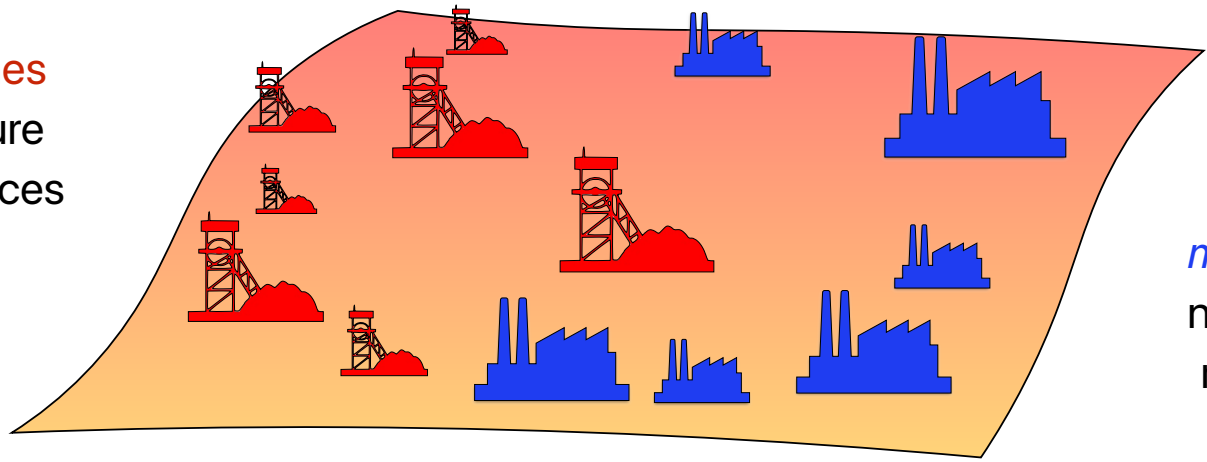


$\mathcal{W}_1$ between discrete color/gradient histograms for image retrieval.

[Credit: Rubner et al., 2000]

Discrete OT: original motivation

n mines
capture
resources

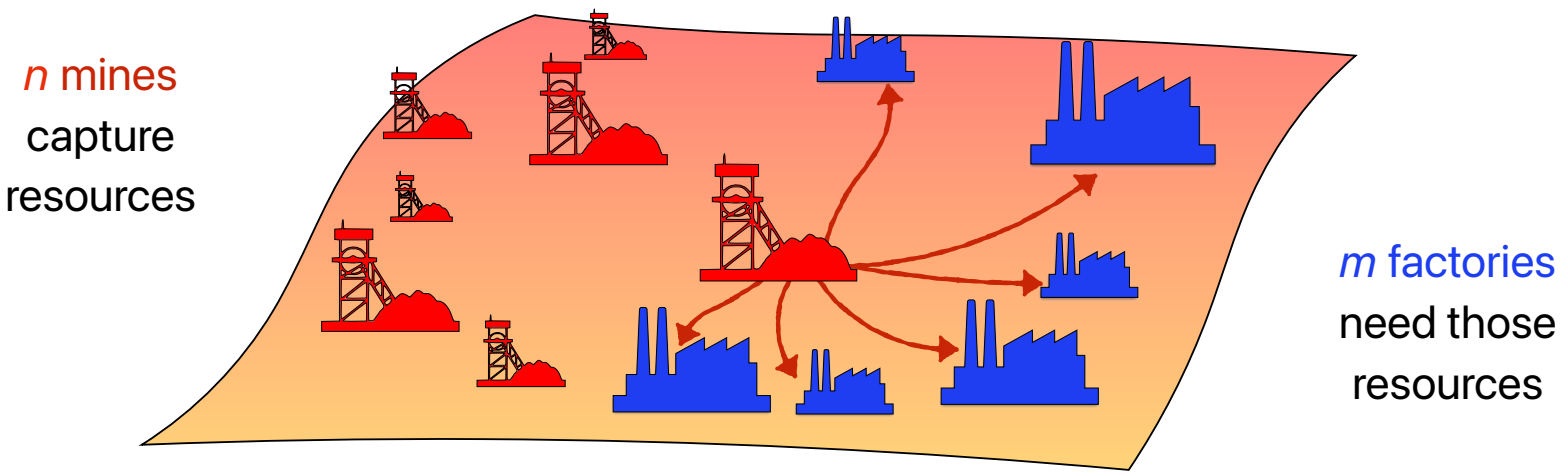


m factories
need those
resources

[Credit: Bunne and Cuturi]

Discrete OT: original motivation

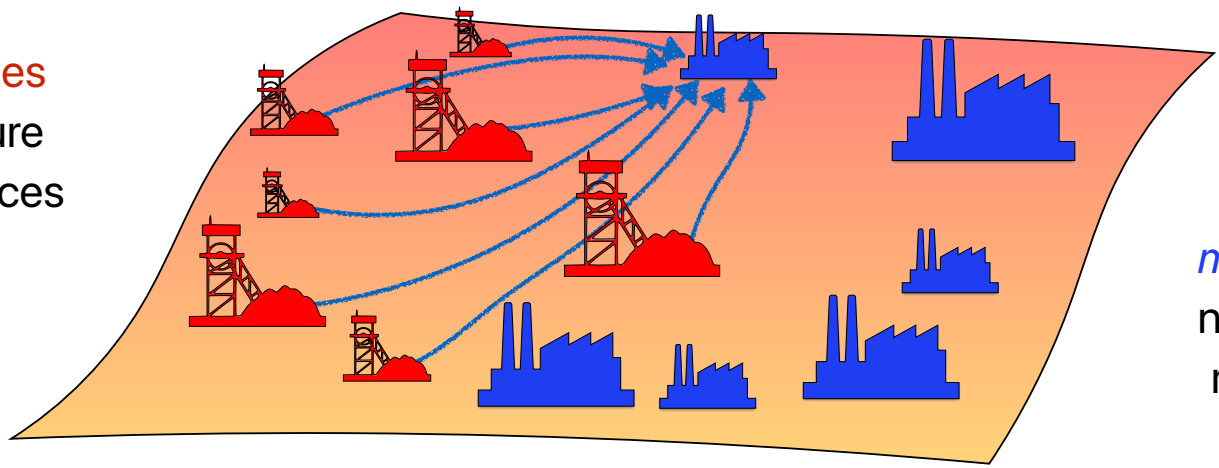
Each of the n mines need to dispatch its production



[Credit: Bunne and Cuturi]

Discrete OT: original motivation

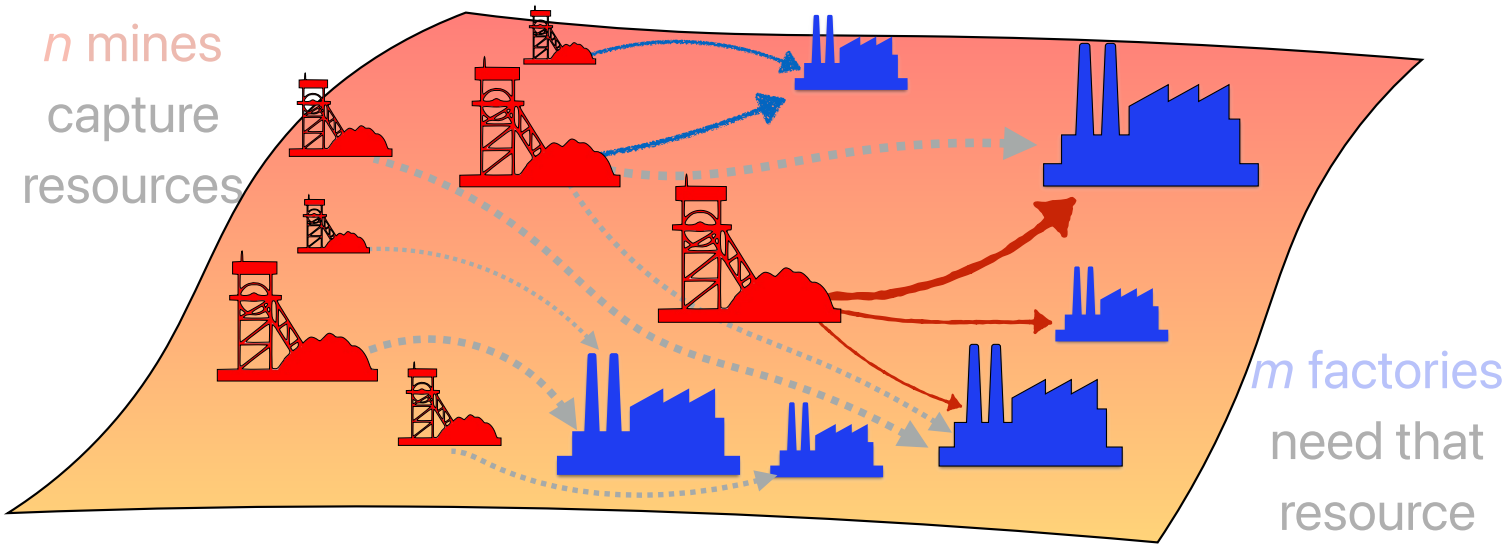
Each of the m factories need to get the resource it needs



[Credit: Bunne and Cuturi]

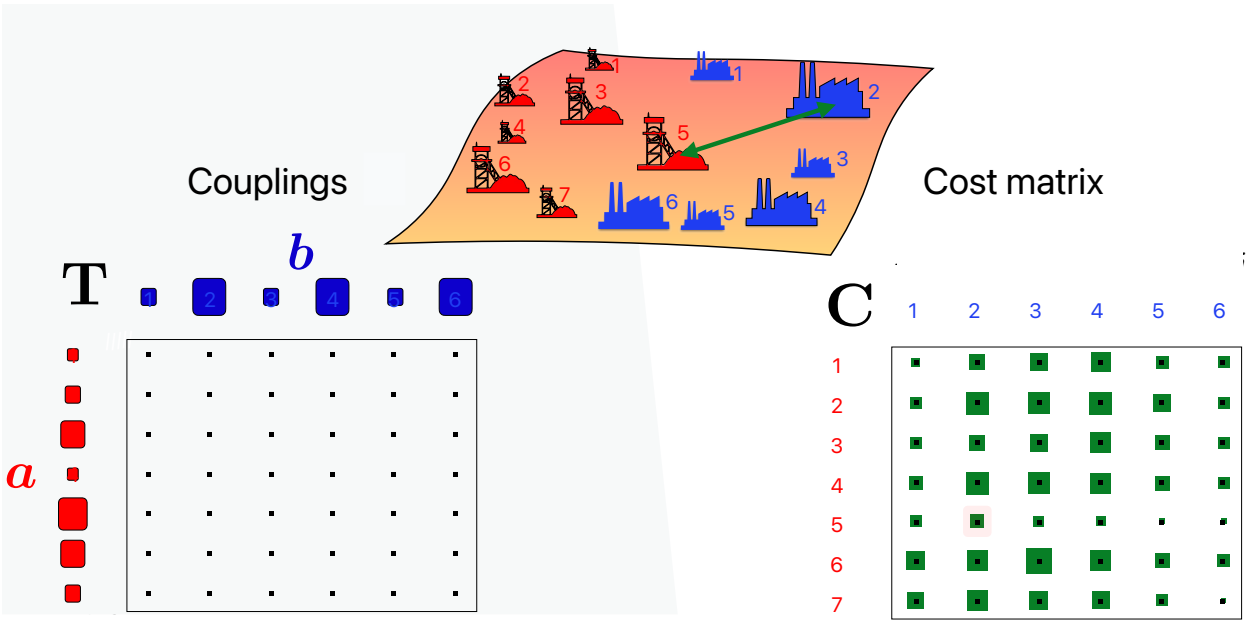
Discrete OT: original motivation

Optimal transport computes the least-costly transport matrix



[Credit: Bunne and Cuturi]

Discrete OT: original motivation



$$\min_{T \in \Pi(a,b)} \langle T, C \rangle_F ,$$

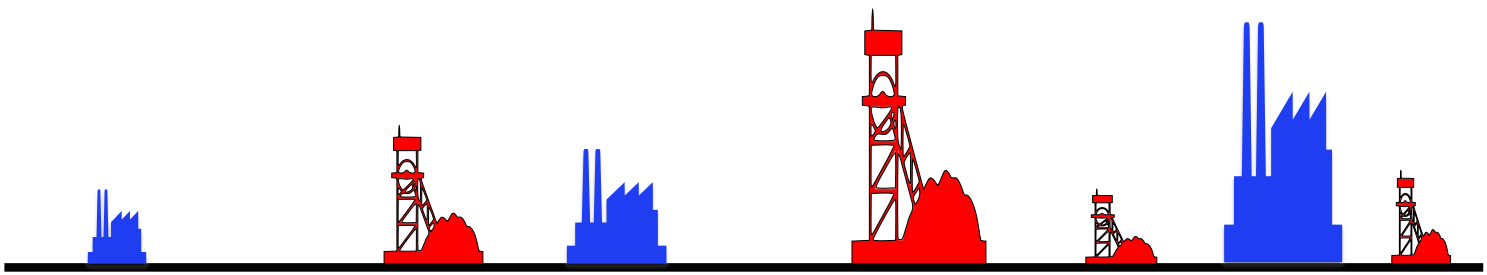
with

$$\Pi(a, b) = \left\{ T \in (\boldsymbol{R}^+)^{n \times m} | T 1_m = a, T^T 1_n = b \right\}$$

[Credit: Bunne and Cuturi]

Algorithms: 1D case

In 1D, the problem can be solved using sorting:
"Repeat until termination: the leftmost mine transfers as much as it can to the leftmost factory still in need of resource."



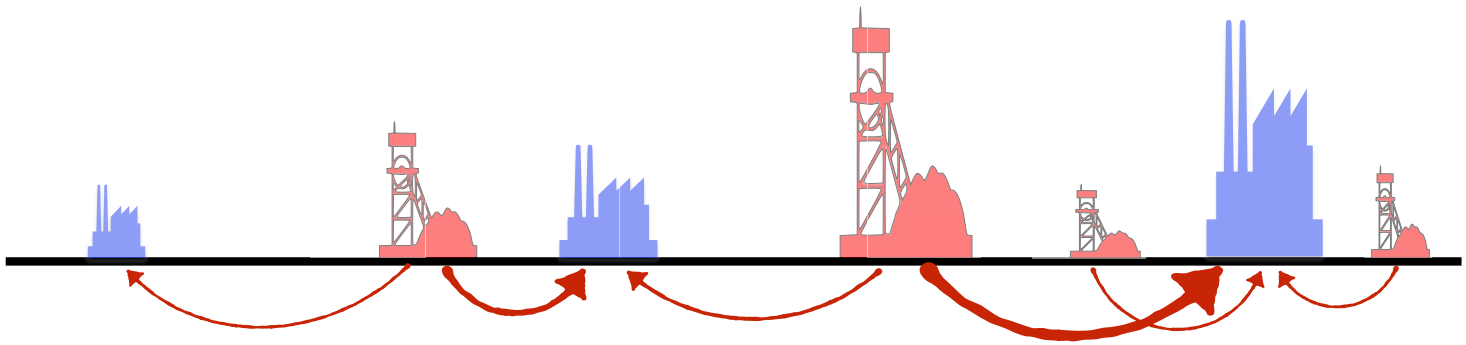
Solution takes $O(n_s \log n_s + n_t \log n_t)$.

NB: works also in the semi-discrete case.

[Credit: Bunne and Cuturi]

Algorithms: 1D case

In 1D, the problem can be solved using sorting:
"Repeat until termination: the leftmost mine transfers as much as it can to the leftmost factory still in need of resource."



Solution takes $O(n_s \log n_s + n_t \log n_t)$.

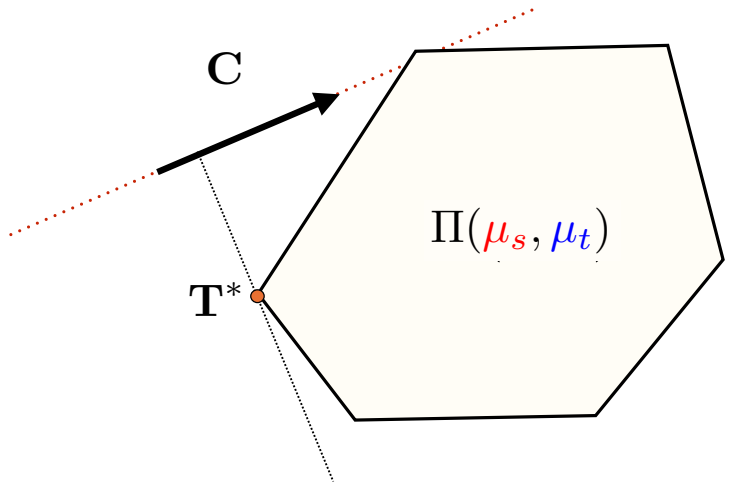
NB: works also in the semi-discrete case.

[Credit: Bunne and Cuturi]

Algorithms: general case

$$\min_{T \in \Pi(\mu_s, \mu_t)} \langle \mathsf{T}, \mathsf{C} \rangle_F ,$$

$$\Pi(\mu_s, \mu_t) = \left\{ \mathsf{T} \in (\boldsymbol{R}^+)^{n_s \times n_t} | \mathsf{T} 1_{n_t} = \mathsf{a}, \mathsf{T}^T 1_{n_s} = \mathsf{b} \right\}$$



A linear program with $n_s n_t$ variables and $n_s + n_t$ constraints. Resolution takes $O((n_s + n_t) n_s n_t \log(n_s + n_t))$ ($\approx O(n^3)$) (ok if $n < 10k$).

Limitations:

- Solution is unstable (non unicity)
- $\mathcal{W}_c(\mu_s, \mu_t)$ is not differentiable

Can be addressed through regularization.

Regularized OT

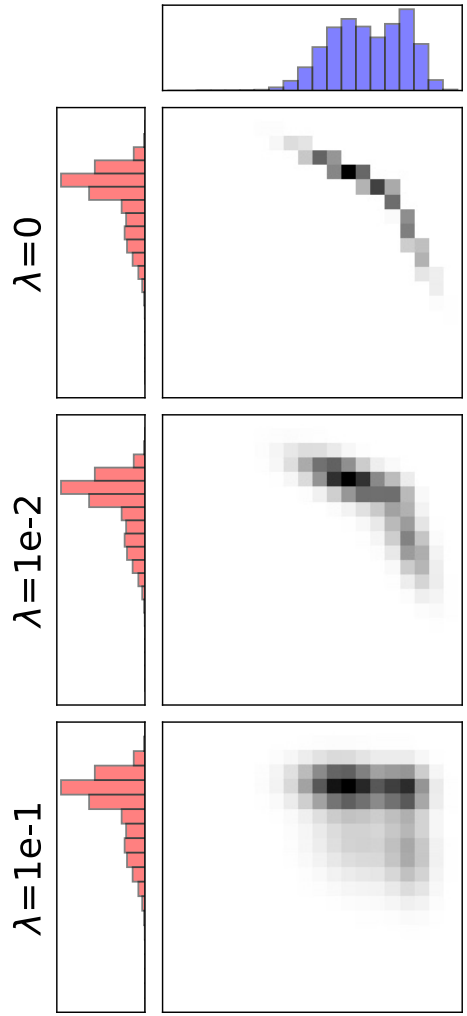
$$\mathrm{T}^*_\lambda = \arg\min_{T \in \Pi(\mu_s, \mu_t)} \langle \mathrm{T}, \mathrm{C} \rangle_F + \lambda \Omega(\mathrm{T})$$

Why regularize?

- Smooth the "distance" estimation

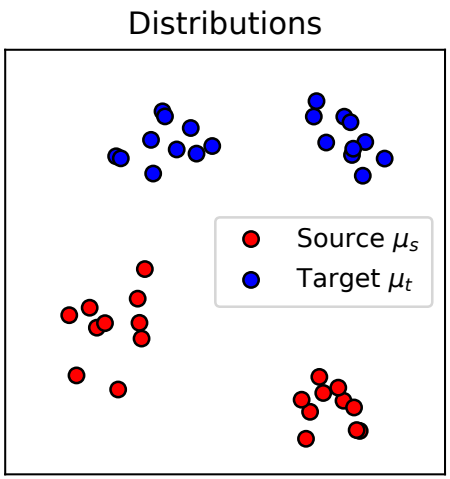
$$\mathcal{W}_{c,\lambda}(\mu_s, \mu_t) = \langle \mathrm{T}^*_\lambda, \mathrm{C} \rangle_F$$

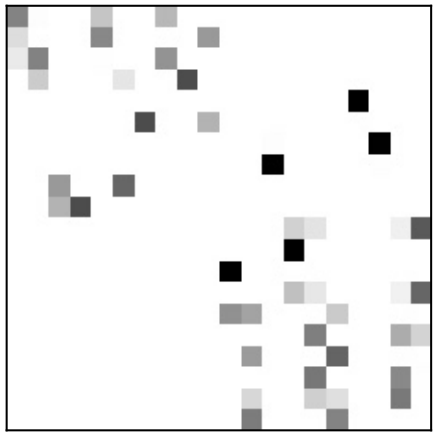
- Encode prior knowledge
- Better posed problem (convex, stability)
- Fast algorithms to solve the OT problem

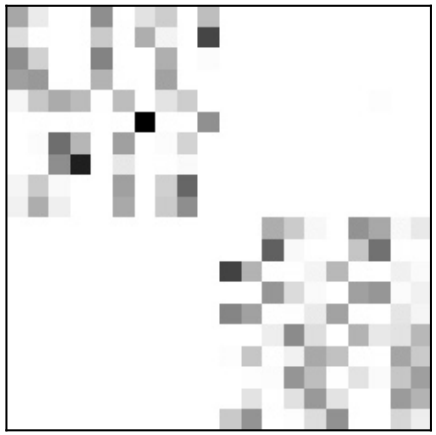


[Credit: Flamary]

Entropy regularized optimal transport (Cuturi, 2013)



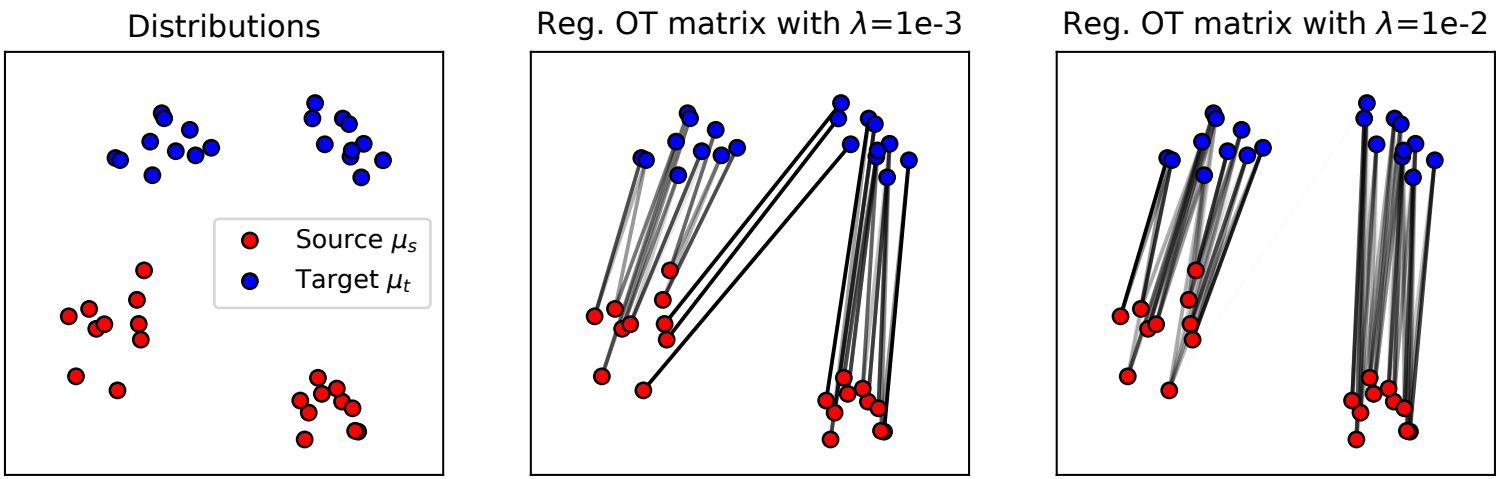




$$\mathrm{T}^*_\lambda = \arg\min_{T \in \Pi(\mu_s, \mu_t)} \langle \mathrm{T}, \mathrm{C} \rangle_F + \lambda \sum_{i,j} T_{ij}(\log T_{ij} - 1)$$

- Regularization term is negative entropy of T (minimum when coupling is uniform).
- Looses sparsity, gains stability.
- Results in a strictly convex optimization problem.
- Loss and OT matrix are differentiable.

Entropy regularized optimal transport (Cuturi, 2013)



$$\mathrm{T}^*_\lambda = \arg\min_{T \in \Pi(\mu_s, \mu_t)} \langle \mathrm{T}, \mathrm{C} \rangle_F + \lambda \sum_{i,j} T_{ij}(\log T_{ij} - 1)$$

- Regularization term is negative entropy of T (minimum when coupling is uniform).
- Looses sparsity, gains stability.
- Results in a strictly convex optimization problem.
- Loss and OT matrix are differentiable.

Solving the entropy regularized problem (Cuturi, 2013)

Lagrangian of the optimization problem

$$\mathcal{L}(\mathrm{T}, \alpha, \beta) = \sum_{i,j} T_{ij} C_{ij} + \lambda T_{ij}(\log T_{ij} - 1) + \alpha^T(\mathrm{T}1_{n_t} - \mathrm{a}) + \beta^T(\mathrm{T}^T 1_{n_s} - \mathrm{b})$$

$$\partial\mathcal{L}(\mathrm{T}, \alpha, \beta)/\partial T_{ij} = C_{ij} + \lambda \log T_{ij} + \alpha_i + \beta_j$$

$$\partial\mathcal{L}(\mathrm{T}, \alpha, \beta)/\partial T_{ij} = 0 \Rightarrow T_{ij} = \exp\left(\frac{\alpha_i}{\lambda}\right) \exp\left(\frac{-C_{ij}}{\lambda}\right) \exp\left(\frac{\beta_j}{\lambda}\right)$$

We therefore have the following property:

$\exists!\mathrm{u} \in (\boldsymbol{R}^+)^{n_s}, \mathrm{v} \in (\boldsymbol{R}^+)^{n_t}$ such that:

$$T^*_\lambda = \mathrm{diag}(\mathrm{u})\mathrm{K}\mathrm{diag}(\mathrm{v}), \text{ with } \mathrm{K} \triangleq \exp(-\mathrm{C}/\lambda).$$

Optimization problem can be solved using Sinkhorn's algorithm.

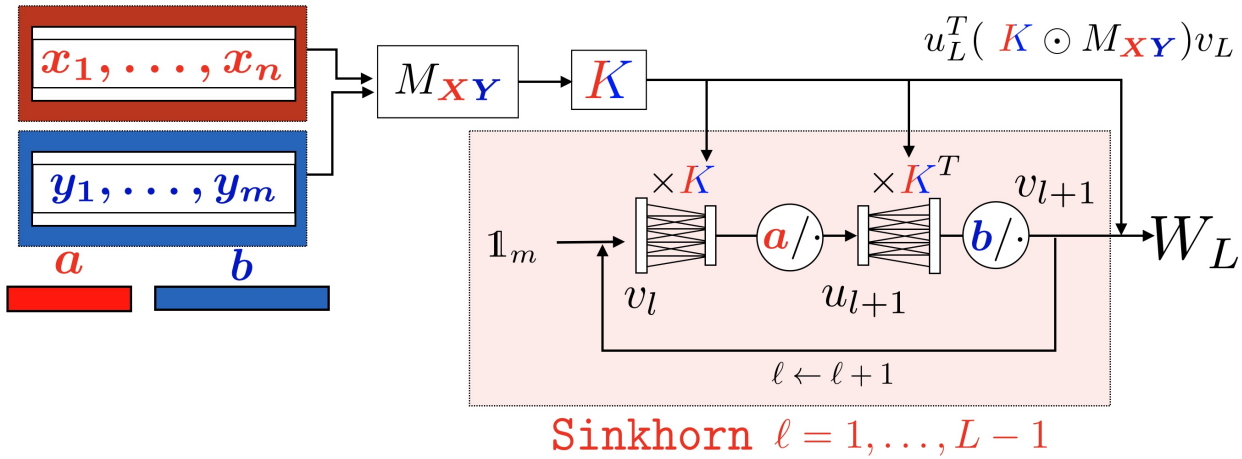
Sinkhorn's algorithm (Cuturi, 2013)

1. Compute $K = \exp(-C/\lambda)$.
2. Initialize v as a vector of ones.
3. Repeat until convergence:
   - $u \leftarrow a \oslash (Kv)$ (*Make the rows sum to* a)
   - $v \leftarrow b \oslash (K^T u)$ (*Make the columns sum to* b)

   ($\oslash$ is element-wise division)

- Very simple algorithm.
- Complexity is $O(kn^2)$ for k iterations.
- Fast implementation in parallel and GPU friendly.
- Fully differentiable

Differentiability of Sinkhorn's algorithm (Cuturi, 2013)



The entire algorithm is a sequence of simple tensor operations. Automatic differentiation frameworks like PyTorch and JAX can trace right through the repeat loop.

One can backpropagate through the Sinkhorn iterations to update a neural network.

Other regularization and algorithms

- Group Lasso (Courty et al., 2016):

$$\Omega(\mathrm{T}) = \sum_{g} \sqrt{\sum_{i,j \in \mathcal{G}_g} T_{i,j}^2}$$

  Promotes group sparsity.

- Frobenius norm (Blondel et al., 2017):

$$\Omega(\mathrm{T}) = \sum_{i,j} T_{i,j}^2$$

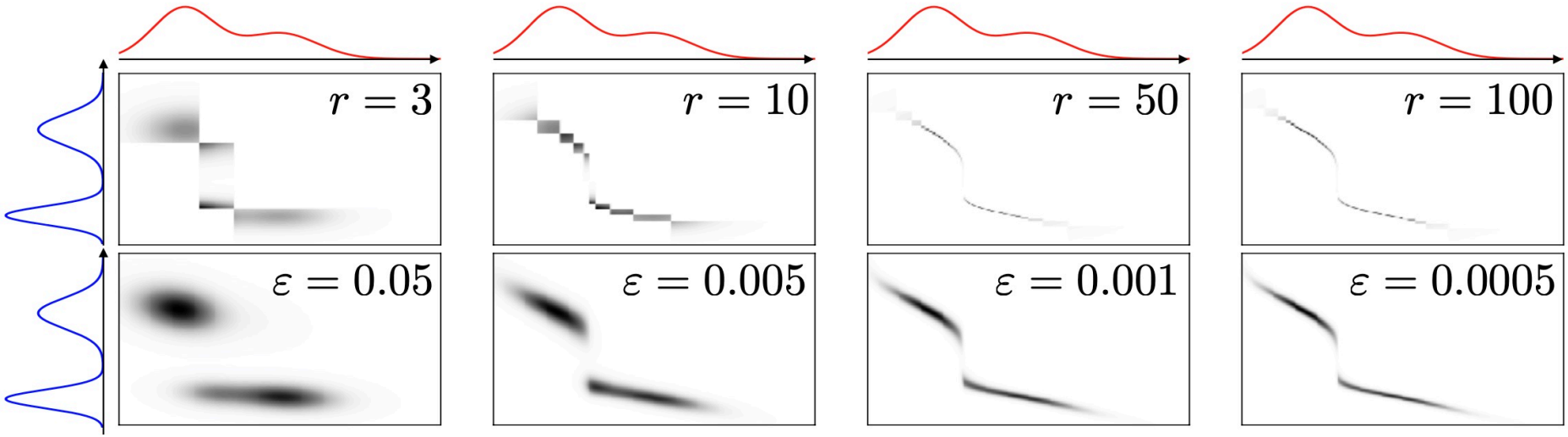
  Strongly convex regularization that keeps sparsity in the solution

- Low-rank constrained OT (Forrow et al., 2019):

$$T = Q\mathrm{diag}(1/g)R^T, Q \in (\boldsymbol{R}^+)^{n_s \times r}, R \in (\boldsymbol{R}^+)^{n_s \times r}, g \in \boldsymbol{R}^+$$

  Leads to fast algorithms (can handle problems with 500k samples)

Low-ranked constrained discrete optimal transport



[Credit: Bunne and Cuturi]

Outline

Further extensions

The Kantorovich formulation can be extended to address more problems:

- Partial OT: only a portion of the mass is required to be transported
- Unbalanced OT: can transport between distributions with different total mass
- Multi-marginal OT: transport between more than two distributions
- Gromov-Wasserstein OT: transport across metric spaces (no cost function can be defined)
- Co-Optimal Transport: transport across samples and features

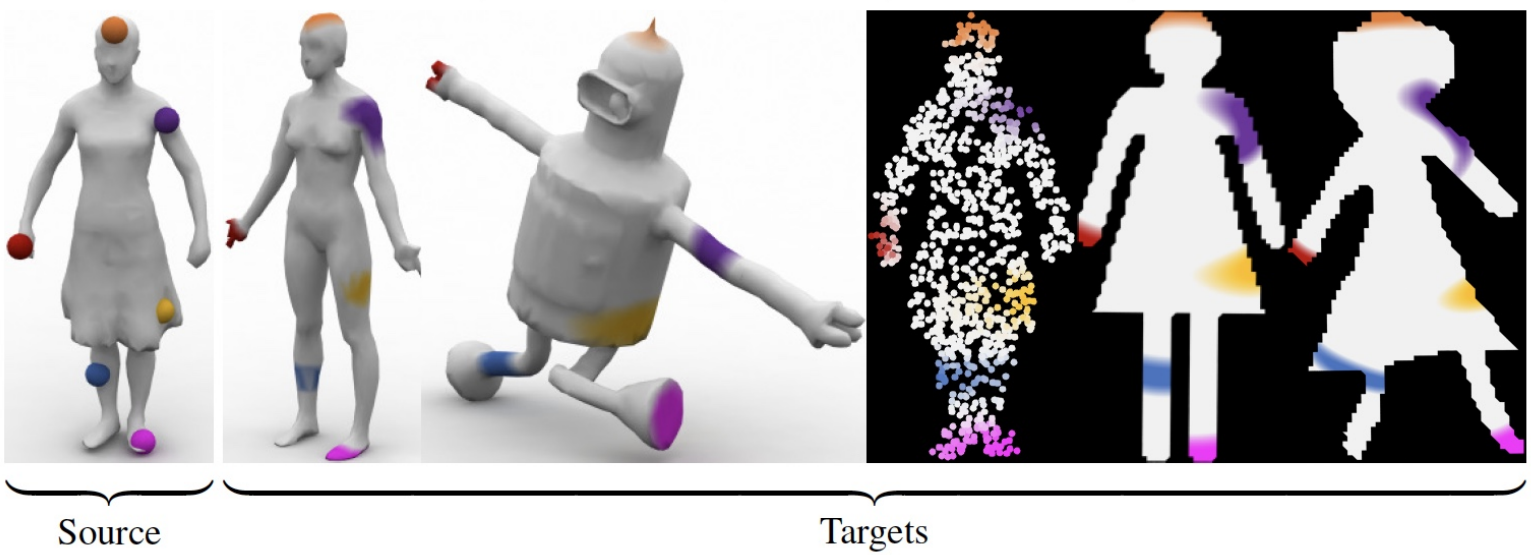
Gromov-Wasserstein OT

Solves:

$$\mathcal{GW}_p(\mu_s, \mu_t) = \left(\min_{\mathbf{T} \in \Pi(\mu_s, \mu_t)} \sum_{i,j,k,l} |D^s_{i,k} - D^t_{j,l}|^p \mathbf{T}_{i,j} \mathbf{T}_{k,l} \right)^{1/p}$$

with $\mu_s = \sum_i a_i \delta_{x^s_i}$ and $\mu_t = \sum_i b_i \delta_{x^t_i}$ and $D^s_{i,k} = ||x^s_i - x^s_k||$, $D^t_{j,l} = ||x^t_j - x^t_l||$

Shape matching between surfaces (Solomon et al., 2016):



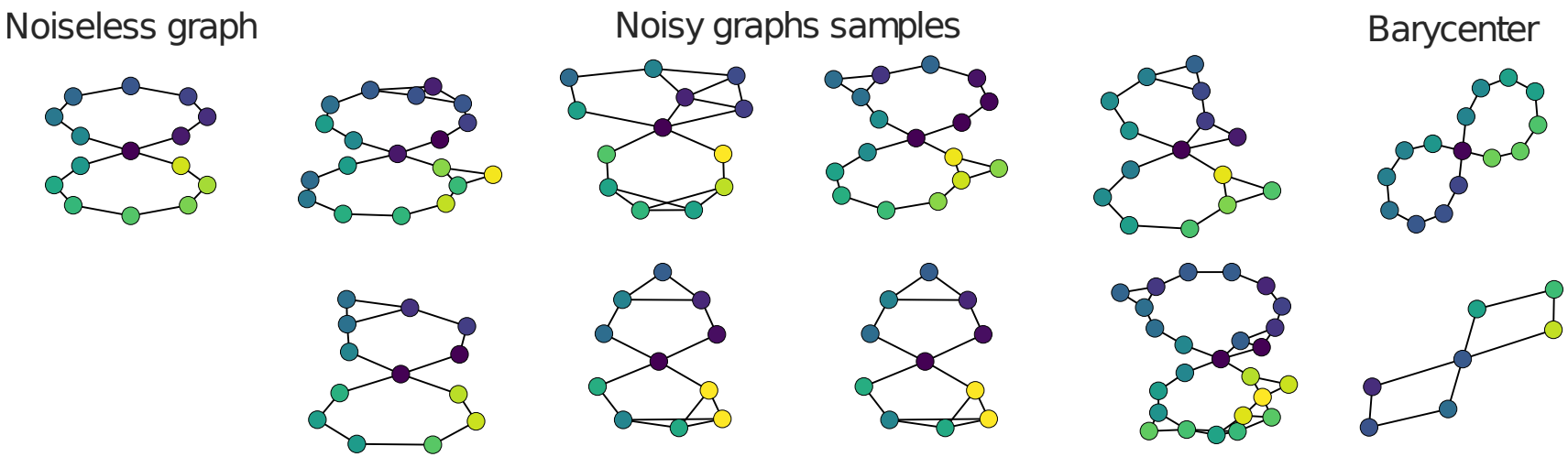
Gromov-Wasserstein OT

Solves:

$$\mathcal{GW}_p(\mu_s, \mu_t) = \left(\min_{\mathbf{T} \in \Pi(\mu_s, \mu_t)} \sum_{i,j,k,l} |D^s_{i,k} - D^t_{j,l}|^p \, T_{i,j} \, T_{k,l} \right)^{1/p}$$

with $\mu_s = \sum_i a_i \delta_{x_i^s}$ and $\mu_t = \sum_i b_i \delta_{x_i^t}$ and $D^s_{i,k} = ||x^s_i - x^s_k||$, $D^t_{j,l} = ||x^t_j - x^t_l||$

Barycenter of graphs: (Vayer et al., 2018):



Outline

Applications

OT provides:

- Principled metrics to compare distributions/measures, in the continuous, discrete and mixed settings.
- Efficient algorithms to compute these metrics and find the optimal transport matrix in the case discrete distributions

Generic applications in ML:

- Distance measures to compare any objects that can be modeled as distributions (point clouds, texts, images, graphs, etc.).
- Loss functions to evaluate or train generative models.
- Algorithms to finding a mapping or correspondance between two spaces.

Outline

Generative Adversarial Networks (GAN)

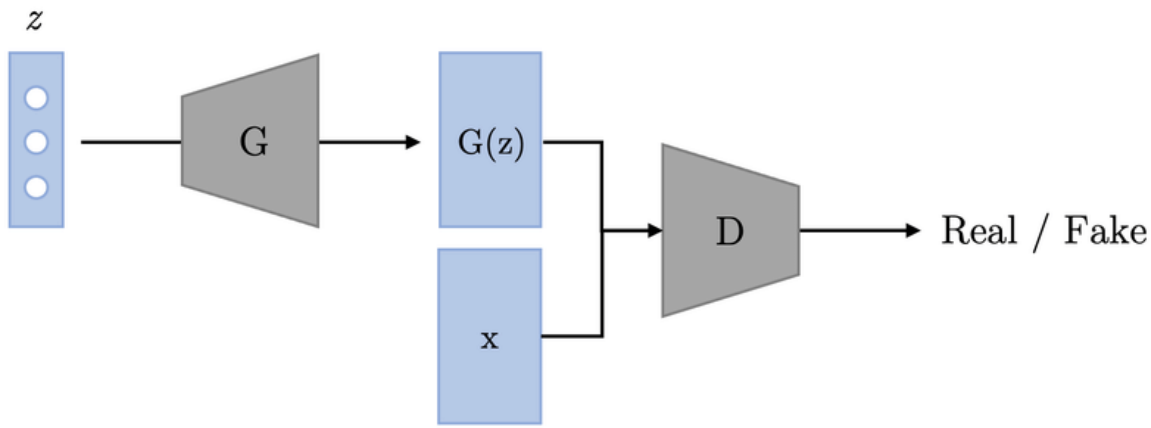
Generative adversarial networks (GAN) (Goodfellow et al., 2014):

- One of the first working image generation model
- Solves:

$$\min_G \max_D E_{x\sim\mu_d}[\log D(x)] + E_{z\sim\mathcal{N}(0,I)}[\log(1 - D(G(z)))],$$

  where G is the generative model and D is a classifier that discriminates generated and true samples.
- Extremely hard to train (vanishing gradients).

Generative Adversarial Networks (GAN)



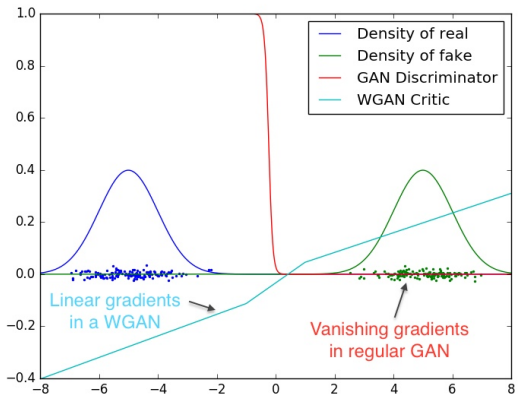
Generative adversarial networks (GAN) (Goodfellow et al., 2014):

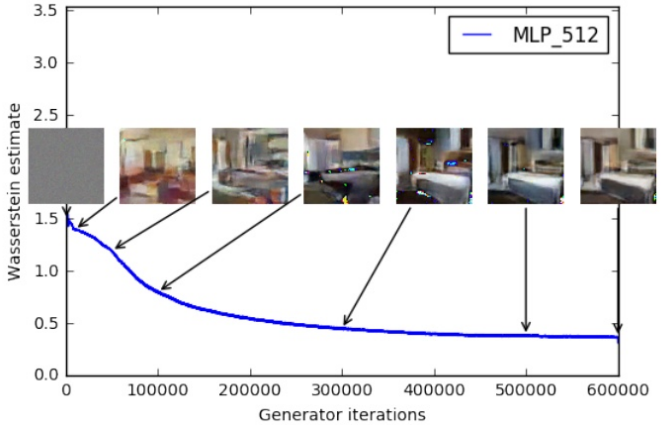
- One of the first working image generation model
- Solves:

$$\min_G \max_D E_{x\sim\mu_d}[\log D(x)] + E_{\mathrm{z}\sim\mathcal{N}(O,\mathrm{I})}[\log(1 - D(G(\mathrm{z})))],$$

  where G is the generative model and D is a classifier that discriminates generated and true samples.
- Extremely hard to train (vanishing gradients).

Wasserstein Generative Adversarial Networks (WGAN)





Wasserstein GAN (Arjovsky et al., 2017):

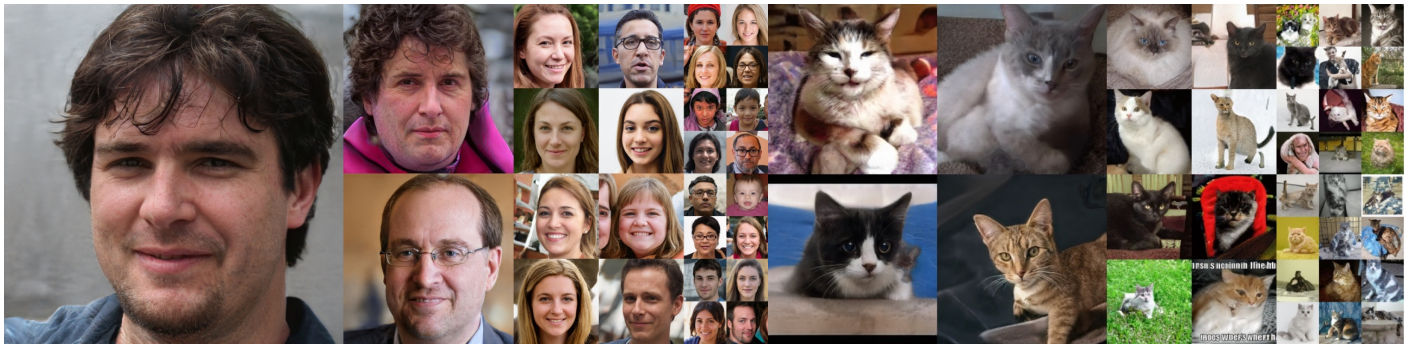
- Solves instead:

$$\min_G \mathcal{W}_1(G\#\mu_z, \mu_d),$$

  i.e., the wasserstein distance between data μ_d and generated data $G\#\mu_z$ distributions.

- No vanishing gradient problems and better convergence in practice
- Need tricks to allow backpropagation (using Kantorovich-Rubinstein duality, not covered in this lecture).
- State of the art for image generation (with many additional tricks) before diffusion models (Karras et al., 2019)

Wasserstein Generative Adversarial Networks (WGAN)



Wasserstein GAN (Arjovsky et al., 2017):

- Solves instead:

$$\min_G \mathcal{W}_1(G\#\mu_z, \mu_d),$$

  i.e., the wasserstein distance between data μ_d and generated data $G\#\mu_z$ distributions.

- No vanishing gradient problems and better convergence in practice
- Need tricks to allow backpropagation (using Kantorovich-Rubinstein duality, not covered in this lecture).
- State of the art for image generation (with many additional tricks) before diffusion models (Karras et al., 2019)

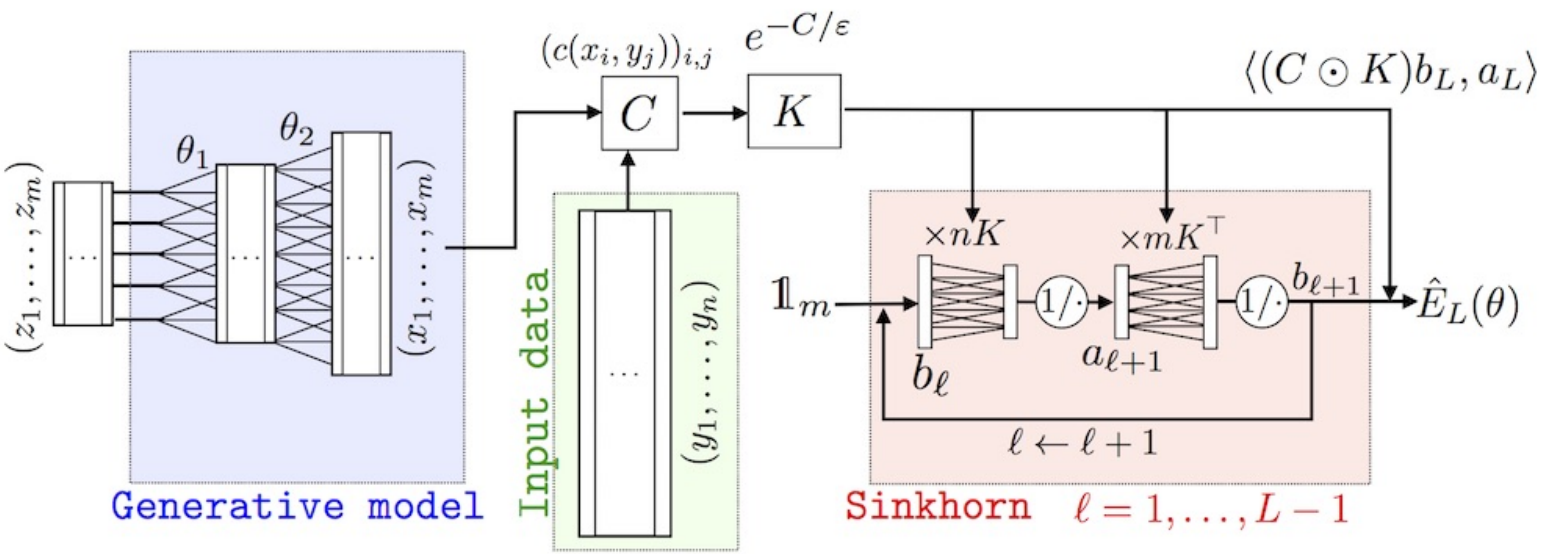
Learning generative models with Sinkhorn (Genevay et al., 2017)

- Use *Sinkhorn divergence* as the loss function:

$$\bar{\mathcal{W}}_{c,\lambda}(\mu_s, \mu_t) = 2\mathcal{W}_{c,\lambda}(\mu_s, \mu_t) - \mathcal{W}_{c,\lambda}(\mu_s, \mu_s) - \mathcal{W}_{c,\lambda}(\mu_t, \mu_t),$$

where $\mathcal{W}_{c,\lambda}(\mu_1, \mu_2) = \langle \mathrm{T}^*_\lambda, \mathrm{C} \rangle_F$. True divergence ($\bar{\mathcal{W}}_{c,\lambda}(\mu, \mu) = 0$) and better statistical property than Wasserstein distance.

- Use Sinkhorn's algorithm and autodiff to train (with mini-batches):



Using Monge maps as generative models

Previous approaches use OT-based distances as loss functions between data and generated samples.

One can instead learn OT maps as generative models directly:

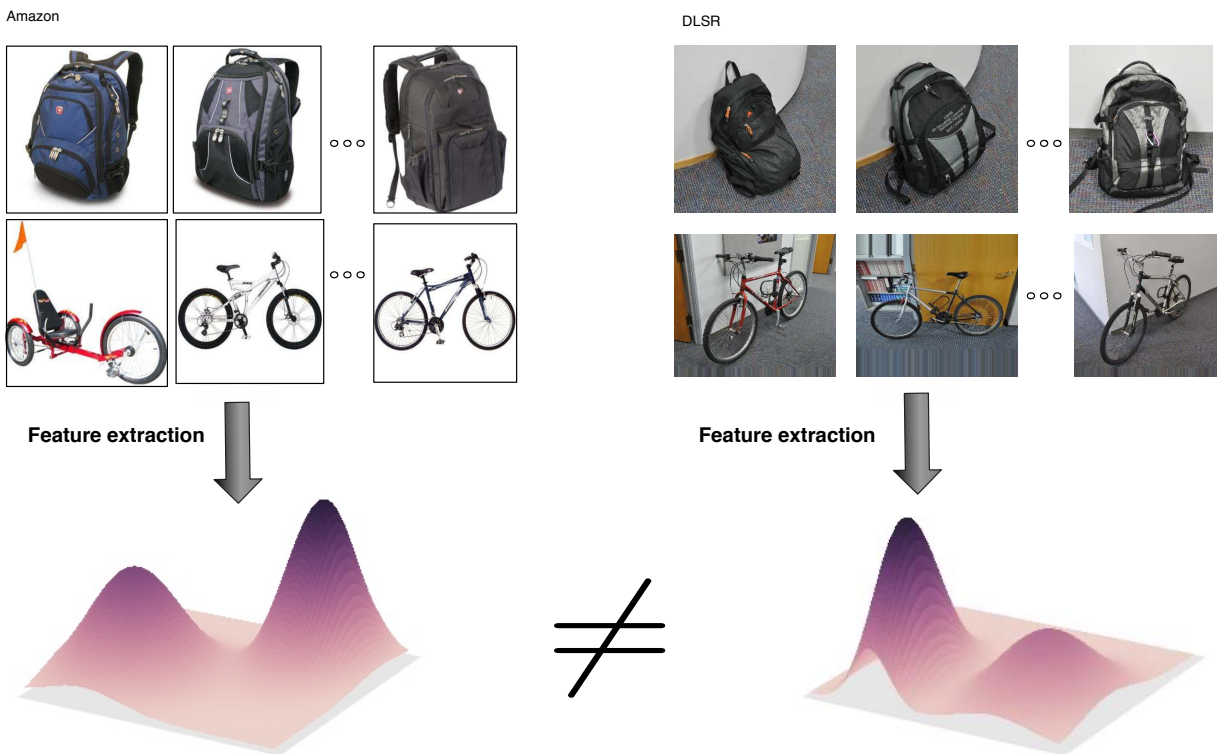
$$T^* = \arg \min_{T : T\#\mu_s \approx \mu_t} \int c(x, T(x)) \mu_s(x) dx$$

(a.k.a. *neural optimal transport* when T is a neural network)

Non trivial problem. Several parameterization of T have been proposed.

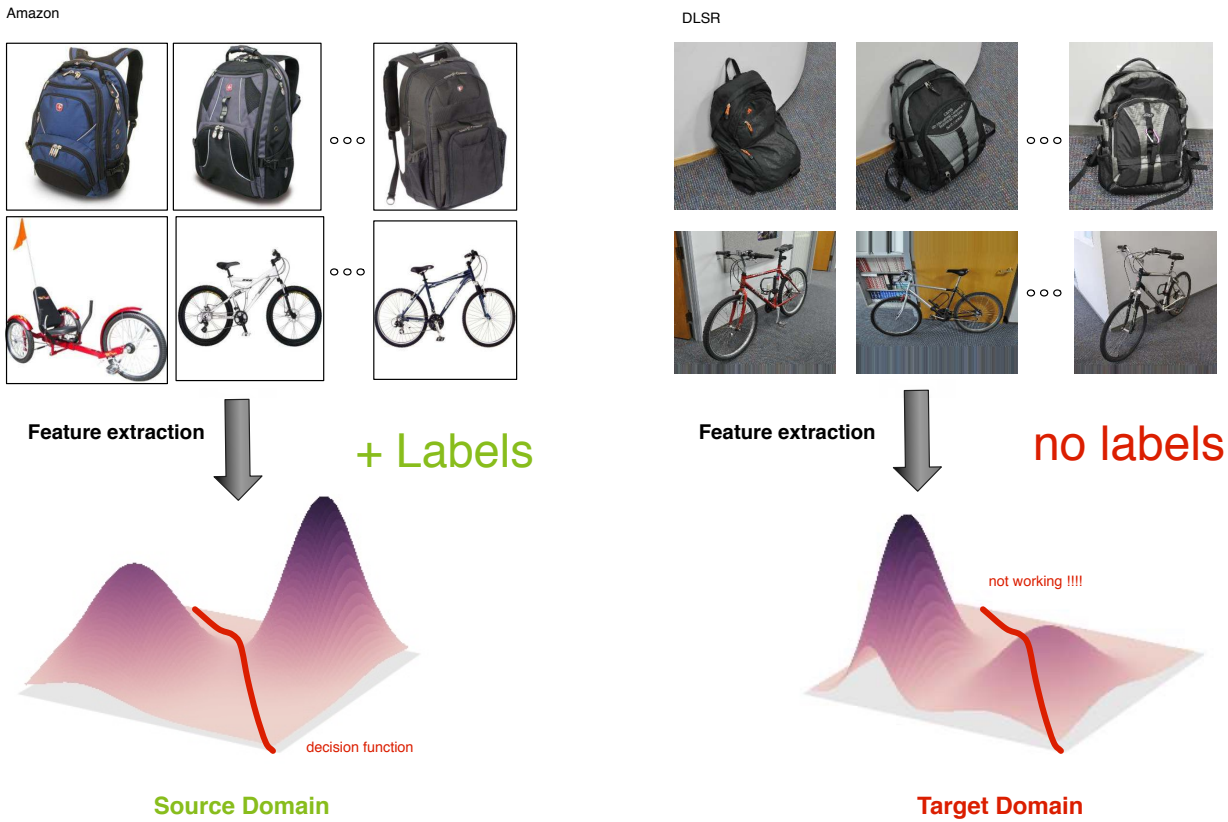
See (Brunne and Cuturi, 2023) for a tutorial on these methods.

Domain adaptation



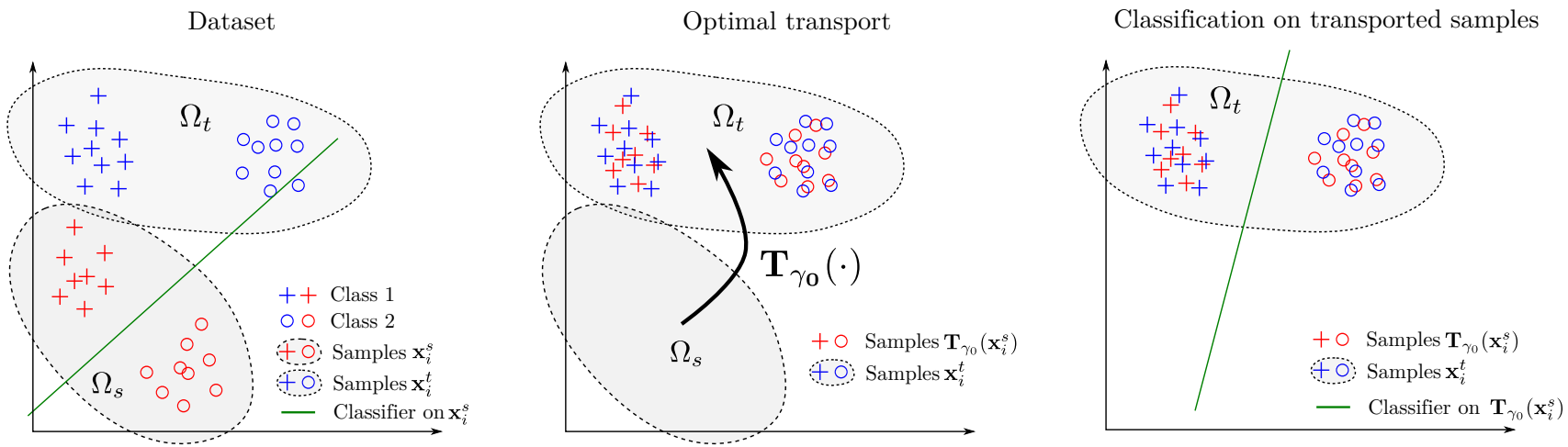
- Classification with data coming from different sources (domains).
- Labels only available in the source domain, and classification is conducted in the target domain.
- Examples: weather conditions, period of time, sensor drift, simulation vs real data, etc.

Domain adaptation



- Classification with data coming from different sources (domains).
- Labels only available in the source domain, and classification is conducted in the target domain.
- Examples: weather conditions, period of time, sensor drift, simulation vs real data, etc.

OT for domain adaptation (Courty et al., 2016)

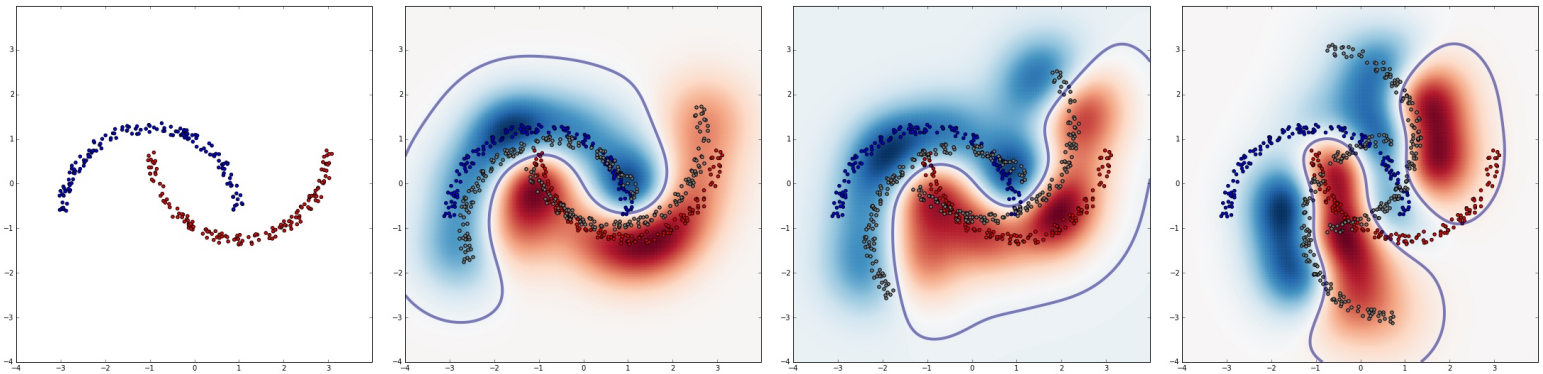


Three steps:

1. Estimate an optimal transport between the distributions
2. Transport the source examples onto the target distribution
   - Using, e.g., barycenters from rows of T^*.
3. Train the model on the transported source examples

OT for domain adaptation

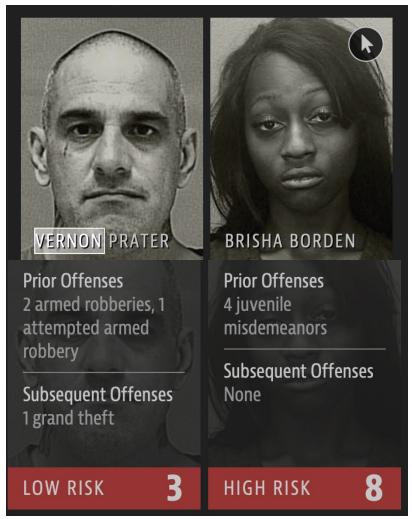
(Courty et al., 2016)



(a) source domain (b) rotation=20° (c) rotation=40° (d) rotation=90°

Target rotation angle	10°	20°	30°	40°	50°	70°	90°
SVM (no adapt.)	0	0.104	0.24	0.312	0.4	0.764	0.828
DASVM [9]	0	0	0.259	0.284	0.334	0.747	0.82
PBDA [22]	0	0.094	0.103	0.225	0.412	0.626	0.687
OT-exact	0	0.028	0.065	0.109	0.206	0.394	0.507
OT-IT	0	0.007	0.054	0.102	0.221	0.398	0.508
OT-GL	0	0	0	0.013	0.196	0.378	0.508
OT-Laplace	0	0	0.004	0.062	0.201	0.402	0.524

Fairness in ML

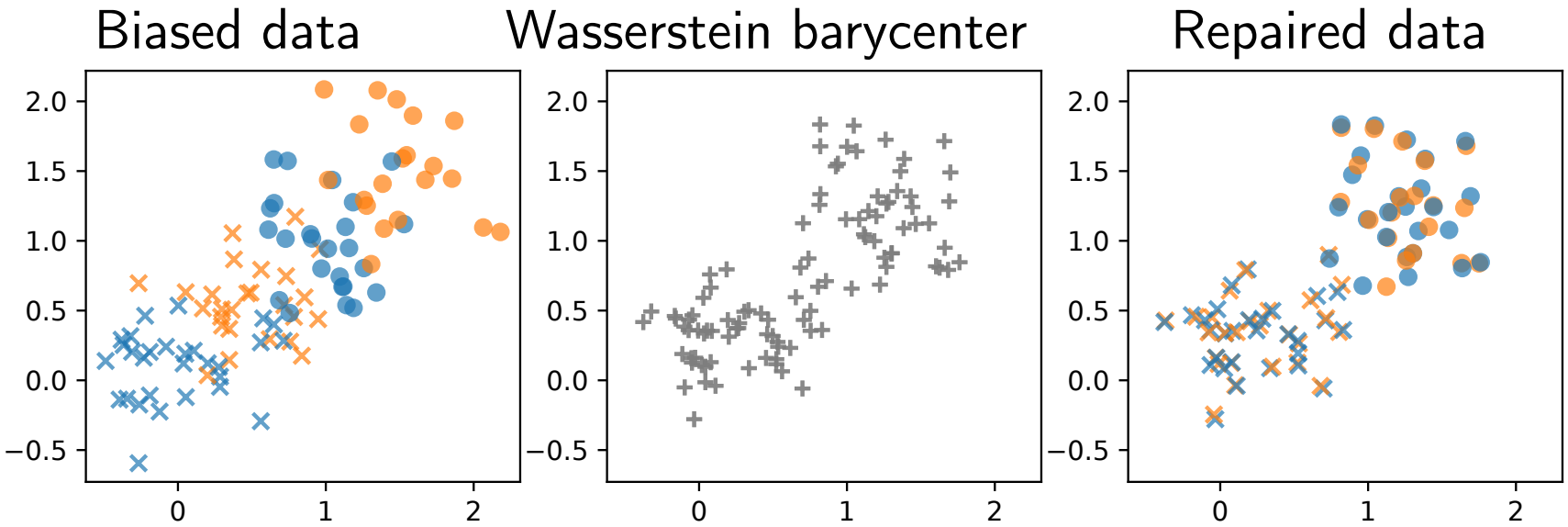


Source: propublica.org

- ML models trained on biased data can have discriminatory effect.
- Fairness in machine learning: make algorithmic decision independent from some variable.
- Formally (del Barrio et al., 2019)
  - $Y \in \{0, 1\}$ the output class, $X \in \boldsymbol{R}^d$ visible attributes, $S \in \{0, 1\}$ protected attribute
  - Find a transformation T_S such that $\mathcal{L}(T_0(X)|S = 0) = \mathcal{L}(T_1(X)|S = 1)$, with $\mathcal{L}$ some loss.

OT for fairness

(del Barrio et al., 2019)



- Split the data according to protected variable S
- Compute the barycenter between the two distributions
- Transport each distribution towards the barycenter (repaired data)
- Train the classifier on the repaired data
- Apply the transport based on S when making a prediction

OT for fairness (del Barrio et al., 2019)

Adult Income dataset: predict if income is $\geq 50K$ per year based on age, education level, capital gain, capital loss, and worked hours per week. Protected variable is gender.

Statistical Model	Error	$\hat{DI}$	CI 95%
Logit	0.2064	0.496	$(0.437, 0.555)$
Random Forests	0.168	0.484	$(0.429, 0.54)$

Statistical Model	Repair	Error	Difference	$\hat{DI}$	CI 95%
Logit	**(A)**	0.218	0.0116	0.937	$(0.841, 1.033)$
Logit	**(B)**	0.2077	0.00128	1	$(0.905, 1.095)$
Logit	**(C)**	0.2132	0.0068	0.94	$(0.842, 1.038)$
Random Forests	**(A)**	0.2272	0.0592	1.1	$(0.976, 1.223)$
Random Forests	**(B)**	0.2045	0.0365	1	$(0.886, 1.114)$
Random Forests	**(C)**	0.2152	0.0472	1.091	$(0.978, 1.203)$

$$\hat{DI} = \frac{P(f(X)=1|S=0)}{P(f(X)=1|S=1)} \ (\approx 1 \text{ means no bias})$$

Content-based image retrieval in digital pathology

See Maxime Amodei's presentation.

Softwares and references

Further references:

- Gabriel Peyré and Marco Cuturi, Computational Optimal Transport, ArXiv:1803.00567, 2018. https://optimaltransport.github.io.
- Charlotte Bunne and Marco Cuturi, "Optimal transport in learning, control, and dynamical systems", ICML Tutorial, 2023.
- A lot of resources on Rémy Flamary's web page: https://remi.flamary.com/.

Softwares:

- POT: Python Optimal Transport (NumPy/SciPy)
- OTT: Optimal Transport Tools (JAX).
- GeomLoss: Geometric loss functions between sampled measures, images and volumes (PyTorch)